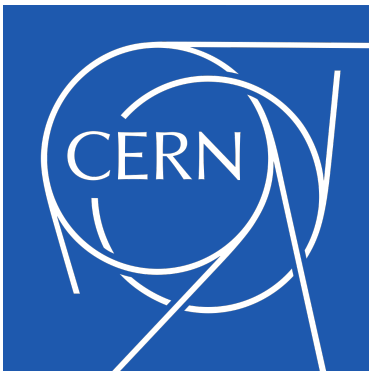



**École Polytechnique Fédérale de Lausanne**

# On Automatic Differentiation: Improving the Unified Framework for Accelerator Beam Physics in MAD-NG

by Antonino Emanuele Scurria

Master Thesis

Approved by the Examining Committee:

Prof. Mark Seidel
Thesis Advisor

Dr. Tatiana Pieloni
Co-Advisor

Dr. Laurent Deniau
Thesis Supervisor

January 10, 2025

*Lausanne, January 10, 2025* Antonino Emanuele Scurria

γλυκύπικρον

# Abstract

The work advances MAD-NG's differential algebra framework by systematically identifying and addressing numerical instabilities encountered in high-order computations and extreme use cases. Through the application of perturbative approaches, novel analytical results have been derived to solve the identified computational problems, improving the numerical accuracy of several functions to a level close to their symbolic computational counterpart.

Furthermore, we will give the quantitative impact of these new results in tables for each of these improved functions, followed by qualitative comments; the C code of these functions will be provided in the Appendix. Through a simple simulation of beam-beam interaction, we will highlight how some of those improvements enhance the accuracy of this particular use case.

MAD-NG's differential algebra framework is considered nowadays to be one of the most powerful and fastest open-source libraries for handling a wide range of problems in physics, typically involving complex behaviors and dynamical systems, like in accelerator beam physics. Consequently, the improvements made by this work offer an elegant, integrated solution that will propagate to a wide range of applications built from the library.

# Contents

# Chapter 1

# Dual Numbers: theory and applications

## 1.1 Overview of Dual Numbers

Dual numbers form a mathematical extension $\mathbb{D}$ of the real $\mathbb{R}$ and complex $\mathbb{C}$ numbers that allow for the representation of infinitesimal quantities. They are defined using a dual unit $\varepsilon$ with the property $\varepsilon^2 = 0$ and $\varepsilon \neq 0$. The dual numbers have applications in fields such as differential calculus, kinematics, and automatic differentiation, where they facilitate calculations of derivatives in an elegant and efficient manner.

**Definition 1.1.1.** *A **dual number** $x \in \mathbb{D}$ is an element of the form*

$$x = a + b\varepsilon,$$

*where $a, b \in \mathbb{C}$, and $\varepsilon$ is a unit with the property $\varepsilon^2 = 0$ and $\varepsilon \neq 0$. The number $a$ is called the **scalar part** of $x$, and the number $b$ is called the **dual part**.*

We define addition, multiplication, and conjugation in the ring $\mathbb{D}$ and prove several properties.

**Definition 1.1.2.** *Let $x = a + b\varepsilon$ and $y = c + d\varepsilon$ be two dual numbers in $\mathbb{D}$.*

- *The **sum** of $x$ and $y$ is given by*

$$x + y = (a + c) + (b + d)\varepsilon.$$

- ***Scalar Multiplication:*** *For any scalar $\alpha \in \mathbb{C}$, the scalar multiplication of $\alpha$ with $x$ is*

$$\alpha \cdot x = (\alpha \cdot a, \alpha \cdot b).$$

- *The **product** of $x$ and $y$ is given by*

$$x \cdot y = (a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon,$$

  *where we use the fact that $\varepsilon^2 = 0$.*

- *The **conjugation** of a dual number $x$ is defined as*

$$\overline{x} = a - b\varepsilon.$$

**Proposition 1.** *The set $\mathbb{D}$ of dual numbers is a commutative ring with respect to addition and multiplication.*

*Proof.* To prove commutativity, associativity, and distributivity, we consider the operations as defined above and verify that they satisfy the axioms of a commutative ring

$$x + y = (a + c) + (b + d)\varepsilon = (c + a) + (d + b)\varepsilon = y + x,$$

and

$$x \cdot y = ac + (ad + bc)\varepsilon = ca + (cb + da)\varepsilon = y \cdot x.$$

The remaining axioms are verified similarly. □

**Proposition 2.** *For any dual number $x = a + b\varepsilon$, the conjugate satisfies*

$$x \cdot x^* = a^2.$$

*Proof.* Since $x^* = a - b\varepsilon$, we have

$$x \cdot x^* = (a + b\varepsilon)(a - b\varepsilon) = a^2 - ab\varepsilon + ab\varepsilon - b^2\varepsilon^2 = a^2.$$

The terms involving $\varepsilon$ cancel out, and $b^2\varepsilon^2 = 0$. □

**Proposition 3.** *If $x = a + b\varepsilon$ with $a \neq 0$, the inverse of $x$ exists and is given by*

$$x^{-1} = \frac{1}{a} - \frac{b}{a^2}\varepsilon.$$

*Proof.* We seek $y = c + d\varepsilon$ such that $x \cdot y = 1$. Expanding $x \cdot y$, we find:

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon.$$

Setting the scalar and dual parts equal to those of $1 + 0\varepsilon$, we obtain $ac = 1$ and $ad + bc = 0$. Solving these equations yields $c = \frac{1}{a}$ and $d = -\frac{b}{a^2}$, giving the result. □

### 1.1.1 Differentiation Using Dual Numbers

A notable application of dual numbers is in automatic differentiation. By evaluating functions at dual number inputs, we can obtain both the function value and its derivative simultaneously.

**Definition 1.1.3.** *Let $f : \mathbb{C} \to \mathbb{C}$ be a differentiable function, and let $x = a + \varepsilon$ be a dual number. Then the **dual extension** of $f$ evaluated at $x$ is given by*

$$f(a + \varepsilon) = f(a) + f'(a)\varepsilon.$$

*Proof.* Expanding $f$ in a Taylor series around $a$ and substituting $a + \varepsilon$ for $x$, we have:

$$f(a + \varepsilon) = f(a) + f'(a) \cdot \varepsilon + \frac{f''(a) \cdot \varepsilon^2}{2!} + \cdots.$$

Since $\varepsilon^2 = 0$, all terms of order $\varepsilon^2$ and higher vanish, leaving

$$f(a + \varepsilon) = f(a) + f'(a)\varepsilon.$$

□

**Example 1.** *Let $f(x) = x^2$. To find $f'(3)$ using dual numbers, consider $x = 3 + \varepsilon$:*

$$f(3 + \varepsilon) = (3 + \varepsilon)^2 = 9 + 6\varepsilon.$$

*Thus, $f(3) = 9$ and $f'(3) = 6$.*

## 1.2 Higher-Order Dual Numbers

To extend the concept of dual numbers to higher orders, we generalize the notion of infinitesimal terms to represent higher derivatives. For a given order $N$, a higher-order dual number encodes information

about a function and its derivatives up to the $N$-th order. This extension is beneficial for automatic differentiation, Taylor series expansions, and computations involving higher-order partial derivatives in multivariable functions.

**Definition 1.2.1.** *Let $N \in \mathbb{N}$ be the order of the dual number. A higher-order dual number $q$ of order $N$ is an ordered tuple:*

$$q = \sum_{i=0}^{N} q_i \varepsilon^i =: (q_0, q_1, q_2, \ldots, q_N),$$

*where $q_0 \in \mathbb{C}$ represents the scalar component, and $q_1, q_2, \ldots, q_N \in \mathbb{C}$ are coefficients associated with infinitesimal terms representing the first, second, and higher-order derivatives up to the $N$-th order.*

The algebra of higher-order dual numbers involves component-wise definitions for addition and multiplication, which are consistent with the rules for derivatives.

**Definition 1.2.2.** *Let $q = (q_0, q_1, \ldots, q_N)$ and $r = (r_0, r_1, \ldots, r_N)$ be two higher-order dual numbers of order $N$.*

- ***Addition:*** *The sum of $q$ and $r$ is given by*

$$q + r = (q_0 + r_0, q_1 + r_1, \ldots, q_N + r_N).$$

- ***Scalar Multiplication:*** *For any scalar $c \in \mathbb{C}$, the scalar multiplication of $c$ with $q$ is*

$$c \cdot q = (c \cdot q_0, c \cdot q_1, \ldots, c \cdot q_N).$$

- ***Multiplication (Leibniz's rule):*** *The product of $q$ and $r$ is defined component-wise by*

$$(q \cdot r)_i = \sum_{k=0}^{i} \binom{i}{k} q_k \, r_{i-k}, \quad i = 0, 1, \ldots, N,$$

  *where $\binom{i}{k} = \frac{i!}{k!(i-k)!}$ is the binomial coefficient.*

This multiplication rule generalizes the product rule for derivatives. For instance, in the case of second-order dual numbers, where $q = (q_0, q_1, q_2)$ and $r = (r_0, r_1, r_2)$, the product components are computed as:

$$
\begin{aligned}
s_0 &= q_0 r_0, \\
s_1 &= q_0 r_1 + q_1 r_0, \\
s_2 &= q_0 r_2 + q_1 r_1 + q_2 r_0.
\end{aligned}
$$

Each term $s_i$ in the product $s = q \cdot r = (s_0, s_1, \ldots, s_N)$ incorporates the appropriate derivative terms according to the Leibniz rule.

### 1.2.1 Multivariate dual numbers

Here we briefly describe dual numbers in the multivariate case: we are going to use an analogy of this kind between the dual numbers and monomials

$$
c + \alpha_x \varepsilon_x + \alpha_y \varepsilon_y + \alpha_{x^2} \varepsilon_x^2 + \alpha_{xy} \varepsilon_x \varepsilon_y + \alpha_{y^2} \varepsilon_y^2 \leftrightarrow c + \alpha_x x + \alpha_y y + \alpha_{x^2} x^2 + \alpha_{xy} xy + \alpha_{y^2} y^2 \tag{1.1}
$$

Here we are creating a bijection between second-order dual numbers with two variables and second-order polynomials with two variables. This is a specific case but one can always associate a multivariate polynomial with a dual number (it is just a matter of representation). We then define, inspired by [2], $P(n,v)$ to be the number of distinct monomials in $v$ variables up to order $n$. This can be shown to be equal to

$$
P(n,v) = \frac{(n+v)!}{n!v!}
$$

Now, as done in [2], assume that these $P$ monomials are arranged in some specific order, indexed by $I$. For each monomial $M$, we define $I_M$ as its position in this ordering. Conversely, we denote by $M_I$ the $I$-th monomial in this ordering. For an index $I$, corresponding to a monomial $M_I = x_1^{i_1} x_2^{i_2} \cdots x_v^{i_v}$, we define $F_I = i_1! i_2! \cdots i_v!$.

**Definition 1.2.3** (Algebraic structure of multivariate dual numbers)**.** *Let* $(a_1, a_2, \ldots, a_P)$ *and* $(b_1, b_2, \ldots, b_P)$ *be two dual numbers of order n in v variables and let* $\alpha \in \mathbb{C}$*, then we define.*

- *The* ***sum***

$$
(a_1, a_2, \ldots, a_P) + (b_1, b_2, \ldots, b_P) = (a_1 + b_1, a_2 + b_2, \ldots, a_P + b_P),
$$

- ***Scalar Multiplication:***

$$\alpha \cdot (a_1, a_2, \ldots, a_P) = (\alpha \cdot a_1, \alpha \cdot a_2, \ldots, \alpha \cdot a_P),$$

- *The* ***product***

$$(a_1, a_2, \ldots, a_P) \cdot (b_1, b_2, \ldots, b_P) = (c_1, c_2, \ldots, c_P),$$

*where the coefficients $c_i$ are defined by:*

$$c_i = F_i \sum_{\substack{0 \leq \nu, \mu \leq P \\ M_\nu \cdot M_\mu = M_i}} \frac{a_\nu \cdot b_\mu}{F_\nu \cdot F_\mu}.$$

**Example 2.** *To clarify the last definition of the product, let us consider the case case with $n = 2$ and $v = 2$. There are $P(2,2) = 6$ monomials:*

$$1, x, y, x^2, xy, y^2.$$

*Using the following ordering and the previous definition of the product, we get:*

$$\begin{aligned}
c_1 &= a_1 \cdot b_1, \\
c_2 &= a_1 \cdot b_2 + a_2 \cdot b_1, \\
c_3 &= a_1 \cdot b_3 + a_3 \cdot b_1, \\
c_4 &= 2\left(\frac{a_1 \cdot b_4}{2} + a_2 \cdot b_2 + \frac{a_4 \cdot b_1}{2}\right), \\
c_5 &= a_1 \cdot b_5 + a_2 \cdot b_3 + a_3 \cdot b_2 + a_5 \cdot b_1, \\
c_6 &= 2\left(\frac{a_1 \cdot b_6}{2} + a_3 \cdot b_3 + \frac{a_6 \cdot b_1}{2}\right).
\end{aligned}$$

Finally, we define an operator that will be responsible of the derivation (thus our space becomes a differential algebra).

**Definition 1.2.4.** *We introduce the derivative operator $\partial_\nu$ on the space of vectors. For a vector containing the derivatives of a function $f$, we define:*

$$\partial_\nu (a_1, a_2, \ldots, a_N) = (c_1, c_2, \ldots, c_N),$$

*where:*

$$c_i = \begin{cases} 0 & \text{if } M_i \text{ has order } n, \\ a_{I_{(M_i \cdot x_\nu)}} & \text{otherwise.} \end{cases}$$

*Basically, the operator $\partial_\nu$ is just shifting the coefficients we have in the vector representing the dual number.*

**Example 3.** *Let us suppose we are dealing with this dual number*

$$c + \alpha_x \varepsilon_x + \alpha_y \varepsilon_y + \alpha_{x^2} \varepsilon_x^2 + \alpha_{xy} \varepsilon_x \varepsilon_y + \alpha_{y^2} \varepsilon_y^2 = (c, \alpha_x, \alpha_y, \alpha_{xx}, \alpha_{xy}, \alpha_{yy})$$

*Then*

$$\partial_x (c, \alpha_x, \alpha_y, \alpha_{xx}, \alpha_{xy}, \alpha_{yy}) = (\alpha_x, \alpha_{xx}, \alpha_{xy}, 0, 0, 0)$$

With this operation, we are now working in a differential algebra (one can prove that the last operation respects the definition of derivative).

## 1.3 Dual Numbers in accelerator beam physics

Dual numbers have assumed an important role in modern accelerator beam physics, enabling precise and efficient computations critical for the design and analysis of particle accelerators. Their integration into computational frameworks, particularly for automatic differentiation, has revolutionized approaches to beam dynamics, transfer maps, and perturbative analysis. In accelerator physics, perturbative methods are frequently employed to study small deviations from reference trajectories or idealized configurations. These methods are vital for understanding how particles respond to imperfections in the accelerator lattice, such as misaligned magnets, energy deviations, or field nonuniformities. Dual numbers naturally align with this framework by encoding these small perturbations into their infinitesimal components, allowing precise calculations of derivatives and sensitivities. This capability eliminates the need for finite-difference methods, which are prone to numerical errors and can become computationally expensive for high-order derivatives. By embedding dual numbers into automatic differentiation tools, perturbative schemes become both more robust and computationally efficient. Dual numbers excel at this type of analysis because they can encode these small perturbations directly in their mathematical structure. This represents a significant advance over traditional finite-difference methods, which suffer from accumulating numerical errors.

Perhaps one of the most striking applications appears in tracking beam distributions in simulations. Imagine trying to not only follow thousands of particles through an accelerator but also understand how slight adjustments to hundreds of magnets might affect their paths. Dual numbers make this possible by

calculating both the particle trajectories and their sensitivity to system changes simultaneously (see GTPSA [9]) .
Leading software platforms in accelerator beam physics —MAD-NG [5], PTC [6], and COSY INFINITY [7]—have embraced dual numbers, integrating them into their simulation framework. This integration represents more than just a technical improvement; it has transformed how physicists approach accelerators' optics design and analysis. The combination of dual numbers with these sophisticated software tools has created a powerful framework that allows researchers to explore accelerator designs with unprecedented precision and efficiency. Through this mathematical innovation, accelerator beam physics has gained not just a new tool, but a new way of thinking about particle beam dynamics and accelerator optimization. Dual numbers have become a very important tool in modern accelerator beam physics , providing a foundation for precise calculations that drive the development of next-generation particle accelerators. Their impact extends beyond mere computational efficiency: they have deepened our analyses of the complex, nonlinear dynamics that govern particle beams.

### 1.3.1 Dual numbers and the Truncated Power Series Algebra

In this section, we focus on the power of this method and the underlying equivalence relation between functions in the 'normal world' and functions in the 'dual number world.' It is important to realize that the functions we commonly use are not divine gifts but rather the results of differential equations. For example, consider the differential equation

$$y' = y \tag{1.2}$$

Its solution is $e^x$, which we call the exponential function. This demonstrates that the "exponential function" and $e^x$ are merely symbols we use to describe the solution of that equation.

Now, ask yourself: can you compute $e^1$ with pen and paper? Likely not. And just as you cannot, neither can our CPUs without additional information. However, if we define

$$e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!} \tag{1.3}$$

you (and the CPU) can compute an approximate value of $e^1$ with as much precision as desired using pen and paper.

This simple example underscores the importance of power series: they are not only a computational tool but also a cognitive tool. By understanding that we can represent analytical functions as power

series, we build the formal relation

$$f(x) \Leftrightarrow \sum_{i=0}^{\infty} \mathscr{C}_i x^i \tag{1.4}$$

This relation is important because is giving us a framework to compute the value of the functions using the algebra of real and complex numbers, something we are really used to (are we able to compute anything without addition, subtraction, multiplication, and division? These operations define algebra itself.). Specifically, if we use Taylor series around a point $x$, we have

$$f(x+h) \Leftrightarrow \sum_{i=0}^{\infty} \frac{f^{(i)}(x)}{i!} h^i \tag{1.5}$$

where $h$ is a small perturbation (depending on the function, the series can converge even with very large perturbations). When transitioning from the 'scalar world' (complex and real algebra) to the 'dual number world,' this approach is extended. For instance,

$$f((a,\varepsilon)) := f(a+\varepsilon) \Leftrightarrow \sum_{i=0}^{\infty} \frac{f^{(i)}(a)}{i!} \varepsilon^i \tag{1.6}$$

If we then take a differential unit $\varepsilon$ with an order of nilpotency $n$, the series is truncated as

$$f((a,\varepsilon)) \Leftrightarrow \sum_{i=0}^{n} \frac{f^{(i)}(a)}{i!} \varepsilon^i \tag{1.7}$$

Thus, in the 'scalar world,' analytical functions correspond to infinite power series, whereas in the 'dual number world,' they correspond to truncated power series. This formalism highlights the power of dual numbers: we will deal with a Truncated Power Series Algebra (TPSA).

In transverse dynamics, for example, we often study the behavior of a beam with a perturbative approach relative to the closed orbit. This requires understanding how phase space is deformed under the influence of various orders of the map we use. Studying the deformation of phase space around the closed orbit of complicated transfer maps becomes a computational challenge. To understand the behavior of our system we need to obtain the derivatives and this is a numerical nightmare even with symbolic computation tools (e.g., Mathematica) that take significant time to evaluate the derivatives and compute their numerical value. As an example, consider the 1-dimensional map $\mathscr{M}(z)$, where $z_f$ is a fixed point, i.e., $z_f = \mathscr{M}(z_f)$(assume we have such a point). Suppose the map $\mathscr{M}$ is composed of common functions as follows:

$$\mathscr{M}(z) = \sinh\left(\tanh\left(\arccos\left(\text{Faddeeva}\left(\frac{1}{z}\right)\right)\right)\right) \tag{1.8}$$

with

$$\text{Faddeeva}(z) = e^{-\frac{1}{z^2}} \left(1 + \frac{2i}{\sqrt{\pi}} \int_0^{1/z} e^{t^2} dt\right) \tag{1.9}$$

To compute the derivatives of this map can be complicated, however, this complexity is greatly reduced if we use the TPSA framework. Representing common functions as summations of monomials, we define

$$T_f((z,\varepsilon)) = \sum_{i=0}^{n} \frac{f^{(i)}(z)}{i!} \varepsilon^i \tag{1.10}$$

For a linearized system, we use a differential unit $\varepsilon$ with a nilpotency equal to 2, i.e.,

$$f(a,\varepsilon) = f(a) + f'(a)\varepsilon \tag{1.11}$$

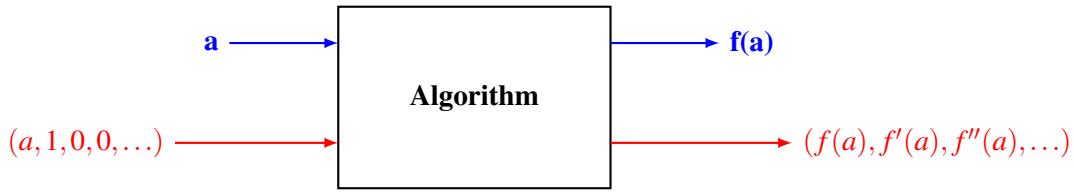
Then, as an example, for the previous map we have

$$\mathscr{M}((z,\varepsilon)) = T_{\sinh}\left(T_{\tanh}\left(T_{\arccos}\left(T_{\text{Faddeeva}}\left(T_{inv}((z,\varepsilon))\right)\right)\right)\right) = \mathscr{M}(z) + \mathscr{M}'(z)\varepsilon \tag{1.12}$$

Effectively, the previously complex problem reduces to a composition of polynomials within the dual number algebra. We can now analyze the stability of the fixed point $z_f$ via $\mathscr{M}'(z_f)$ with ease.

Finally, when transitioning to real-world applications, such as accelerator beam physics , transfer maps often comprise thousands of lines of code applied across a lattice. TPSA enables tracking as many perturbative terms as necessary to study how phase space deforms through transfer map actions nearby the closed orbit. While the cost lies in requiring a stable and concrete truncated power series representation, once established, this representation provides a powerful tool for analyzing systems and their perturbative behavior.

To be more specific, we often encounter situations where we have a complex simulation code consisting of hundreds of thousands of lines. When studying the behavior of a system, we typically want to understand how small perturbations in the input affect the output. To achieve this, we can thus use the algebra we have presented that allows us to explore not just the output for a specific input but also how the output changes in response to slight variations around that input. The following diagram (inspired by [11]) better illustrates this concept.



In this context, $a$ represents the initial point from which we start. Our goal is to investigate how the dynamics of the system are perturbed when we consider points that are near $a$. Instead of running the simulation for a single value of $a$ and getting only $f(a)$ as the result, we push a vector $(a, 1, 0, 0, \ldots)$ through the simulation code. This vector encodes not just the point $a$ but also an infinitesimal perturbation around it. The output then provides more than just $f(a)$; it yields the function's value $f(a)$ as well as its derivatives $f'(a)$, $f''(a)$, and so on.

To achieve this, we replace all standard operations in the simulation code with modified operations we presented within the differential algebra framework. By doing so, we extract a truncated Taylor map that relates the input to the output algebraically, with a specified level of accuracy and order.

This method contrasts with traditional coordinates tracking, where the input and output are connected purely through scalar values. In coordinates tracking, each input corresponds to a single numerical output, providing no direct information about how small changes in the input affect the output. In contrast, by using Truncated Power Series Algebra (TPSA), we obtain a map that algebraically connects the input and output, capturing how the system responds to perturbations. This map can then be used to predict the system's behavior for a range of inputs near $a$ without rerunning the simulation for each new input and to characterize the phase space of our system.

Now, we move to a concrete example about the computation of transfer maps to better enlighten the power of this algebra.

### 1.3.2 The Computation of Transfer Maps

We now focus on the computation of transfer maps in accelerator beam physics using the differential algebra previously discussed, in order to better understand the capabilities of this approach.

Let us consider a simple example, presented in [2] and [17], of midplane motion in a 90° homogeneous bending magnet. The initial displacement is denoted by $x_1$, and the scaled transverse momentum relative to the reference trajectory is defined as $p_1 = \sin(x_1')$ (see Fig. 1.1). We are interested in calculating the final values, $x_2$ and $p_2 = \sin(x_2')$. Since the particle trajectories inside the magnet follow circular arcs, we can directly apply the geometry shown in Fig. 1.1 to derive the following equations:

$$A = R\sin(x_1') = Rp_1, \tag{1.13}$$

$$B = R(1-\cos(x_1')) + x_1 = R\left(1-\sqrt{1-p_1^2}\right) + x_1, \tag{1.14}$$

$$x_2 = A - R(1-\cos x_2') = A - R\left(1-\sqrt{1-p_2^2}\right), \tag{1.15}$$

$$p_2 = \sin(x_2') = -\frac{B}{R}. \tag{1.16}$$

These equations allow for the computation of the final position $x_2$ and transverse momentum $p_2$ in terms of the initial values. By applying differential algebra to these expressions, we can also compute the derivatives of $x_2$ and $p_2$ with respect to $x_1$ and $p_1$. Evaluating these derivatives at $x_1 = 0$ and $p_1 = 0$ gives us the expansion coefficients for the transfer map of the bending magnet.

For clarity, let us explicitly show how $x_2$ and $p_2$ are computed using differential algebra. Following the algebraic rules defined earlier and the indexing order outlined in eq. 1.1, identifying the variable $x$ with the monomial $x$, and $p$ with $y$, we obtain the following:

$$\begin{aligned} x_1 &= (0,1,0,0,0,0), \\ p_1 &= (0,0,1,0,0,0), \\ A &= (0,0,R,0,0,0), \\ B &= (0,1,0,0,0,R). \end{aligned}$$

Using the dual number algebra previously defined, we obtain:

$$x_2 = \left(0,0,R,-\frac{1}{R},0,0,0,\dots\right) = \left(0,\frac{\partial x_2}{\partial x_1},\frac{\partial x_2}{\partial p_1},\frac{\partial^2 x_2}{\partial x_1^2},\frac{\partial^2 x_2}{\partial x_1 \partial p_1},\dots\right), \tag{1.17}$$

$$p_2 = \left(0,-\frac{1}{R},0,0,0,-1,0,\dots\right) = \left(0,\frac{\partial p_2}{\partial x_1},\frac{\partial p_2}{\partial p_1},\frac{\partial^2 p_2}{\partial x_1^2},\frac{\partial^2 p_2}{\partial x_1 \partial p_1},\dots\right). \tag{1.18}$$

This expression agrees with the linear part of the transfer map for the dipole, i.e.,

$$M_{\text{dipole}} = \begin{pmatrix} \cos\left(\frac{L}{R}\right) & R\sin\left(\frac{L}{R}\right) \\ -\frac{1}{R}\sin\left(\frac{L}{R}\right) & \cos\left(\frac{L}{R}\right) \end{pmatrix} = \begin{pmatrix} \frac{\partial x_2}{\partial x_1} & \frac{\partial x_2}{\partial p_1} \\ \frac{\partial p_2}{\partial x_1} & \frac{\partial p_2}{\partial p_1} \end{pmatrix}.$$

For a 90° bending angle, we get the linear matrix:

$$M_{\text{dipole}} = \begin{pmatrix} 0 & R \\ -\frac{1}{R} & 0 \end{pmatrix}.$$

Thus, we notice that, with dual number algebra, we not only recover the linear effect but also the nonlinear ones!

Moreover, if additional optical elements follow this bending magnet, one does not need to start the calculations from $x_1 = (0,1,0,0,0,0)$ and $p_1 = (0,0,1,0,0,0)$. Instead, one can directly use the previously computed values of $x_2$ and $p_2$ as the starting point. This eliminates the need for the often complex concatenation process, resulting in a significant performance and accuracy improvement.

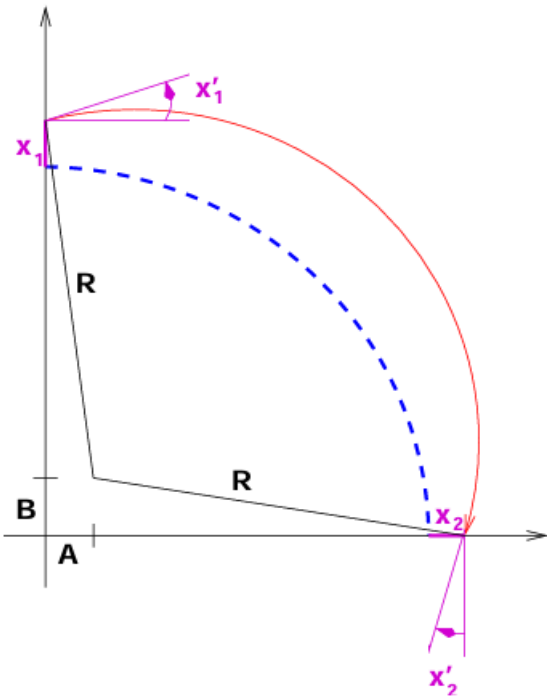



Figure 1.1: Geometry of the particle trajectory in a 90° bending magnet. Figure taken from [17]

### 1.3.3 Numerical maps

Although in the previous example of a 90° bending magnet, we were able to compute everything by hand up to 1st order, this is not the case for higher orders. For most beam physics systems, deriving

analytical formulas for transfer maps is not feasible, except in rare special cases. However, it is possible to compute the relationship between the final and initial coordinates numerically using integration techniques. Essentially, a numerical integration algorithm represents a complex function composed of a finite sequence of elementary operations and functions. Due to its inherent complexity, expressing this function analytically, or differentiating it to high orders with respect to phase-space coordinates or system parameters, is exceedingly difficult.

In this context, the combination of the differential-algebraic (DA) method and the Truncated Power Series Algebra (TPSA) provide a powerful solution by enabling the straightforward computation of high-order maps. This approach involves replacing each elementary operation and function for scalars with their counterparts for TPSA using polymorphic algorithms. One of the main advantages of this approach is that the computational effort for evaluating derivatives remains largely independent of the function's complexity.

Ultimately, using differential algebra, we obtain a Taylor map with as many orders as required.

## 1.4 Applications of Dual Numbers Beyond accelerator beam physics

Dual and hyper-dual numbers have evolved into indispensable tools for exact and efficient computation of derivatives, with wide-ranging applications across science, engineering, and emerging interdisciplinary domains. Their versatility and precision have enabled breakthroughs in fields requiring exact sensitivity analysis and robust numerical methods.

In optimization, dual and hyper-dual numbers excel in tasks demanding high accuracy, such as gradient and Hessian computation. Fike et al. (2011) [18] demonstrated their ability to enhance optimization workflows in complex engineering systems by providing precise derivative information, making them invaluable in areas like aerospace design and machine learning.

In robotics, dual numbers streamline derivative-based calculations critical for control and motion planning. Early applications, such as those by Gu and Luh (1987) [21], showcased their utility in computing manipulators' Jacobians in robotics, while Brodsky and Shoham (1999) [19] extended their use to rigid body dynamics. This improved computational efficiency in kinematic equations has significant implications for autonomous systems, where real-time derivative calculations are essential for adaptive behavior.

Computational mechanics, including structural and fluid dynamics, also benefit from these tools. Dual and hyper-dual numbers enable precise sensitivity analysis and optimization in simulations

governed by partial differential equations (PDEs). Fike and Alonso (2011) [20] demonstrated how hyper-dual numbers enhance second-derivative computations, paving the way for more efficient and accurate simulations in areas such as aerodynamics and material science.

Emerging fields have further underscored the transformative potential of dual numbers. In neuroscience, Wei et al. (2024) [23] applied dual-number-based singular value decomposition to study traveling wave phenomena in the brain, offering novel insights into complex neural dynamics. Similarly, Pal et al. (2024) [24] incorporated hyper-dual numbers into Nonlinearsolve.jl, a high-performance Julia package, to address challenging nonlinear equations. Verified integration of ordinary differential equations (ODEs) and flows using differential algebraic methods on high-order Taylor models, as explored by Berz and Makino [26], highlights the precision and reliability dual numbers bring to dynamics and flow simulations.

These examples illustrate the growing importance of dual numbers as a bridge between theoretical rigor and practical application. Whether in optimization, robotics, computational mechanics, or emerging domains like neuroscience and quantum computing, they offer unmatched precision, efficiency, and adaptability. Their integration into modern software libraries and exploration in advanced computational frameworks ensure dual numbers will continue to play a pivotal role in the future of computation and modeling.

# Chapter 2

# New Series Expansions

## 2.1 Motivation

In this chapter, we present some new analytical series for primitive functions detected as numerically unstable at high orders. The need for these improvements arises from the fact that directly computing these functions within the Truncated Power Series Algebra can lead to significant numerical instabilities, especially for high orders. For example, if we evaluate

$$\operatorname{atan}(x) = \tan^{-1}(x) = \left(\frac{\sin}{\cos}\right)^{-1}(x)$$

directly in the Truncated Power Series Algebra, relying on the primitive functions sin, cos, and the inversion (which are already very stable), we will encounter large instabilities that grow with increasing order. For this reason, we sought a new approach to evaluate these functions. Specifically, functions such as atan, atanh, acot, acoth, and arccosh exhibited instabilities for high orders ($> 6$), while sinc, sinhc, asinc, asinhc and the Faddeeva function showed instabilities even for lower orders.

## 2.2 Atan, Acot, Atanh, Acoth

| Order | Derivative |
|---|---|
| 0 | $\operatorname{atan}(x)$ |
| 1 | $\frac{1}{x^2+1}$ |
| 2 | $-\frac{2x}{(x^2+1)^2}$ |
| 3 | $\frac{2\left(\frac{4x^2}{x^2+1}-1\right)}{(x^2+1)^2}$ |
| 4 | $\frac{24x\left(-\frac{2x^2}{x^2+1}+1\right)}{(x^2+1)^3}$ |
| 5 | $\frac{24\left(\frac{16x^4}{(x^2+1)^2}-\frac{12x^2}{x^2+1}+1\right)}{(x^2+1)^3}$ |
| 6 | $\frac{240x\left(-\frac{16x^4}{(x^2+1)^2}+\frac{16x^2}{x^2+1}-3\right)}{(x^2+1)^4}$ |
| 7 | $\frac{720\left(\frac{64x^6}{(x^2+1)^3}-\frac{80x^4}{(x^2+1)^2}+\frac{24x^2}{x^2+1}-1\right)}{(x^2+1)^4}$ |
| 8 | $\frac{40320x\left(-\frac{16x^6}{(x^2+1)^3}+\frac{24x^4}{(x^2+1)^2}-\frac{10x^2}{x^2+1}+1\right)}{(x^2+1)^5}$ |
| 9 | $\frac{40320\left(\frac{256x^8}{(x^2+1)^4}-\frac{448x^6}{(x^2+1)^3}+\frac{240x^4}{(x^2+1)^2}-\frac{40x^2}{x^2+1}+1\right)}{(x^2+1)^5}$ |

Table 2.1: Derivatives of $\operatorname{atan}(x)$ up to the 9-th order

Here we are going to present and prove some new analytical series for the functions atan, acot, atanh, acoth. We put all these functions together because their series and the derivations are very similar.
Let us start first with some definitions.

**Definition 2.2.1** (Pascal's triangle diagonals)**.** *We define the n-th number of the d-simplex (d dimensions) as:*

$$\mathscr{T}^d(n) := \begin{cases} 1 & \text{if } d = 0 \\ \frac{(n)\times(n+1)\times\ldots\times(n+d-1)}{(d)!} & \text{if } d \geq 1 \end{cases}$$

There are a lot of interesting relations for this numbers. For instance, here we provide the simplest property that will be used later.

**Lemma 2.2.2.** *The n-th number of the d-simplex can be alternatively defined as*

$$\mathscr{T}^d(n) := \binom{n+d-1}{d}$$

*Proof.* Simple calculation. □

We now define the double factorial as it will be used

**Definition 2.2.3** (Double Factorial)**.** *The double factorial of a non-negative integer n, denoted by n!!, is defined as the product of all the integers from n down to 1 that have the same parity (odd or even) as n. Formally:*

$$n!! = \begin{cases} n \times (n-2) \times (n-4) \times \cdots \times 4 \times 2, & \text{if } n \text{ is even}, \\ n \times (n-2) \times (n-4) \times \cdots \times 3 \times 1, & \text{if } n \text{ is odd}. \end{cases}$$

*Additionally, by convention:*

$$0!! = 1 \quad \text{and} \quad (-1)!! = 1.$$

Now we introduce another auxiliary function to have a more compact formula

$$f(n) = (n-1) \times ((n+1) \mod 2) + 1$$

Thus we can finally write the generic derivative of $\operatorname{atan}(x)$.

**Theorem 2.2.4.** *The n-th derivatives of* atan *reads*

$$\operatorname{atan}^{(n)}(x) = \frac{(n-1)! \left[ x^{((n+1) \mod 2)} (-1)^{n+\lfloor \frac{n-3}{2} \rfloor} f(n) + \sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor} (-1)^{n+i+1} 2^{n-1-2i} \mathscr{T}^i(n-2i) \frac{x^{n-1-2i}}{(x^2+1)^{\left\lfloor \frac{n-1-2i}{2} \right\rfloor}} \right]}{(x^2+1)^{\lceil \frac{n+1}{2} \rceil}} \tag{2.1}$$

*Proof.* To prove the previous relation we will follow the easy path of induction. At first, we observe that this relation holds for the previously shown cases in the table of derivatives. Then we are gonna prove by induction the relation diving the cases in odd and even integers. In this proof we define

$$\alpha_i^n := (-1)^{n+i+1} 2^{n-1-2i} \mathscr{T}^i(n-2i)$$

We start with an odd $n$.

**Case 1: $n$ odd**

We start by assuming that eq. 2.1 is valid and then we will show that this implies the validity for $n+1$.

In this case we can rewrite eq. 2.1 as

$$atan^{(n)}(x) = (n-1)! \left[ \frac{(-1)^{n+\frac{n-3}{2}}}{(x^2+1)^{\frac{n+1}{2}}} + \sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right]$$

We then focus on the second term to obtain an interesting relation. Indeed deriving we obtain

$$\frac{\partial}{\partial x} \left( \sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right) = \sum_{i=0}^{\frac{n-3}{2}} \alpha_i^n \left( \frac{2x^{n-2i}(i-n)}{(x^2+1)^{n-i+1}} + \frac{x^{n-2i-2}(n-2i-1)}{(x^2+1)^{n-i}} \right)$$

From this we see that, summing the monomials with the same orders, we get

$$\frac{x^{n-2i-2}(n-2i-1)}{(x^2+1)^{n-i}} \left( (n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n \right)$$

Thus we need to check that

$$(n-1)!(n-2i-1)\alpha_i n + 2(i+1-n)\alpha_{i+1}^n = (n!)\alpha_{i+1}^{n+1}$$

To have a better expression for the left-hand side we compute

$$\frac{\alpha_i^n}{\alpha_{i+1}^n} = -4\frac{(i+1)(n-i-1)}{(n-2i-1)(n-2i-2)}$$

Thus the left-hand side reads

$$(n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n = \alpha_{i+1}^n \left( -4\frac{(i+1)(n-i-1)}{(n-2i-2)} + 2(i+1-n) \right)$$

Writing now explicitly $\alpha_{i+1}^n$ we get

$$(n-2i-1)\alpha_i^n + 2(i+1-n)\alpha_{i+1}^n = \frac{-2n(n-i-1)}{n-2i-2}(-1)^{n+i+2}2^{n-2i-3}\mathscr{T}^{i+1}(n-2i-2)$$

Now writing explicitly also $\alpha_{i+1}^{n+1}$

$$\alpha_{i+1}^{n+1} = (-1)^{n+i+3}2^{n-2i-2}\mathscr{T}^{i+1}(n-2i-1)$$

we see that we only need to check

$$\frac{\mathscr{T}^{i+1}(n-2i-2)\frac{-2n(n-i-1)}{n-2i-2}}{n\mathscr{T}^{i+1}(n-2i-1)} = 1$$

and, using the definition 2.2.1 and the expression through the binomial coefficient, this easily follows.

Now we are left with proving the corner cases, i.e. $i = 0$ and $i = \frac{n-3}{2}$. In the first case, we easily see

$$(n-1)!\alpha_i^n \frac{2x^{n-2i}(i-n)}{(x^2+1)^{n-i+1}}\bigg|_{i=0} = -(n!)\frac{2x^n}{(x^2+1)^{n+1}} = -(n!)\alpha_0^{n+1}\frac{2x^n}{(x^2+1)^{n+1}}$$

that is we have an equality of coefficients for this fraction of monomials. For the case $i = \frac{n-1}{3}$ we will need to take into account the derivative of the piece outside the summation of eq. 2.1 $\frac{(-1)^{n+\frac{n-3}{2}}}{(x^2+1)^{\frac{n+1}{2}}}$ and to sum it with the contribution coming from the summation. In the end we get

$$(n-1)!\frac{x}{(n+1)}(x^2+1)^{-\frac{n+3}{2}}\left(2\alpha^n_{\frac{n-3}{2}} - (n+1)\right)$$

and we need to check that this is equal to the part outside the summation of eq. 2.1 computed with n+1. Using the previous properties this is easily checked.

**Case 2: $n$ even**

Now we start to prove the case with $n$ even. In this case the formula eq. 2.1 reads

$$atan^{(n)}(x) = (n-1)!\left[\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}} + \sum_{i=0}^{\frac{n-3}{2}-\frac{1}{2}} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}}\right]$$

We immediately see that we need to check only the corner cases since the relation between $\alpha_i^n$, $\alpha_{i+1}^n$ and $\alpha_{i+1}^{n+1}$ is still valid. The case $i = 0$ is as before. We now need to check $\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}}$. Derivating this part of the equation will give a monomial with a numerator of 0 degree and another monomial that will be summed with the contribution coming from the summation. The first term is easily checked since we just need to compute it and compare it with the same monomial of the equation for the derivative of order $n+1$ (the term outside the summation). We thus get

$$\frac{\partial}{\partial x}\left(\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}}\right) = \frac{(-1)^{\frac{3n-4}{2}}n}{(x^2+1)^{\frac{n}{2}+1}} + \frac{(-1)^{\frac{3n-4}{2}}2nx^2(-\frac{n}{2}-1)}{(x^2+1)^{\frac{n}{2}+2}}$$

One can easily check that computing the term of the same order of $\frac{(-1)^{\frac{3n-4}{2}}(n!)}{(x^2+1)^{\frac{n}{2}+1}}$ for the formula of degree $n+1$ will have the same coefficient. We thus miss the last piece. To prove that the second piece obtained through the derivation of $\frac{n(-1)^{n+\frac{n-3}{2}-\frac{1}{2}}}{(x^2+1)^{\frac{n+1}{2}+\frac{1}{2}}}$ summed with the coefficient obtained through the

derivation of the summation part will yield the correct coefficient $\alpha_{i+1}^{n+1}$. To prove this we notice that

$$n(-1)^{\frac{3n}{2}} = \alpha^n_{\frac{n-3}{2}+1}$$

. So, since we know that the relation $(n-1)!((n-2i-1)\alpha_i n + 2(i+1-n)\alpha^n_{i+1}) = (n!)\alpha^{n+1}_{i+1}$, we just need to check that in our case

$$(i+1-n) = -(1+\frac{n}{2})$$

and substituting $i = \frac{n-3}{2} + \frac{1}{2}$ we achieve the result we wanted. □

Now we will prove that formula eq. 2.1 can be rewritten more simply.

**Corollary 2.2.5.** *Defining the coefficients*

$$\alpha_i^n := (-1)^{n+i+1} 2^{n-1-2i} \mathscr{T}^i(n-2i)$$

*formula eq. 2.1 can be rewritten as*

$$atan^{(n)}(x) = (n-1)! \left( \sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} \alpha_i^n \frac{x^{n-1-2i}}{(x^2+1)^{n-i}} \right) \tag{2.2}$$

*Proof.* The proof is easy indeed if we compute

$$\lfloor \frac{n-1-2i}{2} \rfloor + \lceil \frac{n+1}{2} \rceil = n-i$$

. Moreover, we now need to prove that $x^{((n+1) \mod 2)}(-1)^{n+\lfloor \frac{n-3}{2} \rfloor} f(n)$ in eq. 2.1 can be written as $\alpha_{\lfloor \frac{n-3}{2} \rfloor + 1}$. When $n$ is even we already know that this is true since we used it in the proof of 2.2.4. We then need just to prove it when $n$ is odd. In this case, we see that

$$x^{((n+1) \mod 2)}(-1)^{n+\lfloor \frac{n-3}{2} \rfloor} f(n) = (-1)^{\frac{3n-3}{2}}$$

and

$$\begin{aligned} \alpha^n_{\lfloor \frac{n-3}{2} \rfloor + 1} = & \\ &= \alpha^n_{\frac{n-1}{2}} \\ &= (-1)^{\frac{3n+1}{2}} 2^0 \mathscr{T}^{\frac{n-1}{2}}(1) \\ &= (-1)^{\frac{3n-3}{2}} \times 1 \end{aligned}$$

The last piece to be checked is that the numerator does not have any power of $x$. This is easily checked

$$x^{n-1-2i}\big|_{i=\frac{n-1}{2}} = x^0$$

For completeness, we write the proof also for the case of $n$ even. In this case, we have

$$x^{((n+1) \mod 2)}(-1)^{n+\lfloor\frac{n-3}{2}\rfloor} f(n) = x(-1)^{\frac{3n-4}{2}} n$$

and

$$\begin{aligned}
\alpha^n_{\lfloor\frac{n-3}{2}\rfloor+1} &= \\
&= \alpha^n_{\frac{n-2}{2}} \\
&= (-1)^{\frac{3n}{2}} 2^1 \mathscr{T}^{\frac{n-2}{2}}(2) \\
&= (-1)^{\frac{3n-4}{2}} 2 \binom{\frac{n}{2}}{\frac{n-2}{2}} \\
&= (-1)^{\frac{3n-4}{2}} 2 \frac{n}{2} \\
&= (-1)^{\frac{3n-4}{2}} n
\end{aligned}$$

And in the end, we check that

$$x^{n-2i-1}\big|_{i=\frac{n-2}{2}} = x^1$$

□

Thus we have some interesting results:

**Corollary 2.2.6.** *The Taylor expansion of* atan *around x reads:*

$$\mathscr{T}_{\text{atan}(x)}(y) = \text{atan}(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor\frac{n-3}{2}\rfloor+1} (-1)^{n+i+1} 2^{n-1-2i} \mathscr{T}^i(n-2i) x^{n-1-2i}\right]}{n(x^2+1)^{n-i}} y^n \tag{2.3}$$

*Proof.* It follows directly from thm. 2.2.4 using the definition of the Taylor Series. □

Trivially one can get the expansion of acot around $x$ in the same way, obtaining (only a minus sign difference):

**Theorem 2.2.7.** *The Taylor expansion of* acot *around x reads:*

$$\mathscr{T}_{\operatorname{acot}(x)}(y) = \operatorname{acot}(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \mathscr{T}^i (n-2i) x^{n-1-2i}\right]}{n(x^2+1)^{n-i}} y^n \tag{2.4}$$

*Proof.* Same proof as theorem 2.2.4 □

One can extend these formulas also to the hyperbolic counterparts to obtain:

**Theorem 2.2.8.** *The Taylor expansion of* atanh *around x reads:*

$$\mathscr{T}_{\operatorname{atanh}(x)}(y) = \operatorname{atanh}(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \binom{n-i-1}{i} x^{n-1-2i}\right]}{n(x^2-1)^{n-i}} y^n \tag{2.5}$$

*Proof.* Same proof as theorem 2.2.4 □

**Theorem 2.2.9.** *The Taylor expansion of* acoth *around x reads:*

$$\mathscr{T}_{\operatorname{acoth}(x)}(y) = \operatorname{acoth}(x) + \sum_{n=1}^{\infty} \frac{\left[\sum_{i=0}^{\lfloor \frac{n-3}{2} \rfloor + 1} (-1)^{n+i} 2^{n-1-2i} \binom{n-i-1}{i} x^{n-1-2i}\right]}{n(x^2-1)^{n-i}} y^n \tag{2.6}$$

*Proof.* Same proof as theorem 2.2.4 □

### 2.2.1 Alternative derivation

Another way to find alternative formulas is to use the complex formulation of the previous functions. Let's take atan again as an example. We then have that:

$$\begin{aligned}
\frac{d^n}{dx^n}\operatorname{atan}(x) &= \frac{d^n}{dx^n}\frac{i}{2}log\left(\frac{x-i}{x+i}\right) \\
&= \frac{1}{2i}\frac{d^{n-1}}{dx^{n-1}}\left(\frac{1}{x-i}-\frac{1}{x+i}\right) \\
&= \frac{(-1)^{n-1}(n-1)!}{2i}\left(\frac{1}{(x-i)^n}-\frac{1}{(x+i)^n}\right) \\
&= \frac{(-1)^{n-1}(n-1)!}{2i}\cdot\frac{(x+i)^n-(x-i)^n}{(x^2+1)^n} \\
&= \frac{(-1)^{n-1}(n-1)!}{2i}\cdot\frac{\sum_{k=0}^{n}\binom{n}{k}x^{n-k}\left(i^k-(-i)^k\right)}{(x^2+1)^n} \\
&= \frac{(n-1)!}{(x^2+1)^n}\sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\binom{n}{2k+1}x^{n-2k-1}(-1)^{n+k-1}
\end{aligned} \tag{2.7}$$

This formula may seem very similar to the previous one, but there is a substantial difference: we notice that here we are summing powers of $x$ without dividing them by powers of $(x^2+1)$. Additionally, the coefficient multiplying the powers of $x$ is different. Thanks to these two differences, we can derive a new formula for the binomial coefficient. Indeed, we know that this identity must hold, as both formulas are correct:

$$\sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\left(\binom{n}{2k+1}-2^{n-1-2k}\binom{n-i-k}{i}(x^2+1)^k\right)\frac{x^{n-2k-1}}{(x^2+1)^n}=0$$

Expanding now $(x^2+1)^k$ we find that $\forall k$ and $\forall n$ (with $k\leq\lfloor\frac{n-1}{2}\rfloor$):

$$\binom{n}{2k+1}=\sum_{i=0}^{\lfloor\frac{n-1}{2}\rfloor-k}(-1)^i 2^{n-1-2(k+i)}\binom{n-(k+i)-1}{k+i}\binom{k+i}{k} \tag{2.8}$$

This is indeed a formula (we do not know if it is new) for the binomial coefficients for odd numbers.

Now, we can follow the exact same procedure for acot. In this case we get :

$$\begin{aligned}\frac{d^n}{dx^n}\operatorname{acot}(x) &= \frac{d^n}{dx^n}\frac{i}{2}log\left(\frac{x-i}{x+i}\right)\\ &= -\frac{d^n}{dx^n}\frac{1}{2i}log\left(\frac{x-i}{x+i}\right)\\ &\vdots\\ &= \frac{(n-1)!}{(x^2+1)^n}\sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\binom{n}{2k+1}x^{n-2k-1}(-1)^{n+k} \quad (1,3)\end{aligned} \tag{2.9}$$

We then proceed to obtain the other 2 alternative expansions for atanh and acoth:

$$\begin{aligned}\frac{d^n}{dx^n}\operatorname{atanh}(x) &= \frac{d^n}{dx^n}\frac{1}{2}log\left(\frac{1+x}{1-x}\right) \qquad if \quad |x|<1\\ &= \frac{1}{2}\frac{d^{n-1}}{dx^{n-1}}\left(\frac{1}{1+x}+\frac{1}{1-x}\right)\\ &= \frac{(-1)^{n-1}(n-1)!}{2i}\left(\frac{1}{(1+x)^n}-\frac{}{(x-1)^n}\right)\\ &= \frac{(-1)^{n-1}(n-1)!}{2}\cdot\frac{(x-1)^n-(x+1)^n}{(x^2-1)^n}\\ &= \frac{(-1)^{n-1}(n-1)!}{2}\cdot\frac{\sum_{k=0}^{n}\binom{n}{k}(x)^{n-k}\left((-1)^k-1\right)}{(x^2-1)^n}\\ &= \frac{(n-1)!}{(x^2-1)^n}\sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\binom{n}{2k+1}x^{n-2k-1}(-1)^{n+k} \quad (1,3)\end{aligned} \tag{2.10}$$

and

$$\begin{aligned}\frac{d^n}{dx^n}\operatorname{acoth}(x) &= \frac{d^n}{dx^n}\frac{1}{2}log\left(\frac{1+x}{1-x}\right) \qquad if \quad |x|>1\\ &\vdots\\ &= \frac{(n-1)!}{(x^2-1)^n}\sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\binom{n}{2k+1}x^{n-2k-1}(-1)^{n+k} \quad (1,3)\end{aligned} \tag{2.11}$$

## 2.3 Acosh

We now analyze acosh(x) to find another series to be applied in our differential algebra of truncated series. Even in this case, the formula was obtained through a deductive process and then was inductively proved.

| Order | Derivative |
|---|---|
| 0 | $\operatorname{acosh}(x)$ |
| 1 | $\frac{1}{\sqrt{(x-1)(x+1)}}$ |
| 2 | $-\frac{\frac{1}{x+1}+\frac{1}{x-1}}{2\sqrt{(x-1)(x+1)}}$ |
| 3 | $\frac{\frac{3}{(x+1)^2}+\frac{2}{(x-1)(x+1)}+\frac{3}{(x-1)^2}}{4\sqrt{(x-1)(x+1)}}$ |
| 4 | $-\frac{3\left(\frac{5}{(x+1)^3}+\frac{3}{(x-1)(x+1)^2}+\frac{3}{(x-1)^2(x+1)}+\frac{5}{(x-1)^3}\right)}{8\sqrt{(x-1)(x+1)}}$ |
| 5 | $\frac{3\left(\frac{35}{(x+1)^4}+\frac{20}{(x-1)(x+1)^3}+\frac{18}{(x-1)^2(x+1)^2}+\frac{20}{(x-1)^3(x+1)}+\frac{35}{(x-1)^4}\right)}{16\sqrt{(x-1)(x+1)}}$ |
| 6 | $-\frac{15\left(\frac{63}{(x+1)^5}+\frac{35}{(x-1)(x+1)^4}+\frac{30}{(x-1)^2(x+1)^3}+\frac{30}{(x-1)^3(x+1)^2}+\frac{35}{(x-1)^4(x+1)}+\frac{63}{(x-1)^5}\right)}{32\sqrt{(x-1)(x+1)}}$ |
| 7 | $\frac{45\left(\frac{231}{(x+1)^6}+\frac{126}{(x-1)(x+1)^5}+\frac{105}{(x-1)^2(x+1)^4}+\frac{100}{(x-1)^3(x+1)^3}+\frac{105}{(x-1)^4(x+1)^2}+\frac{126}{(x-1)^5(x+1)}+\frac{231}{(x-1)^6}\right)}{64\sqrt{(x-1)(x+1)}}$ |
| 8 | $-\frac{315\left(\frac{429}{(x+1)^7}+\frac{231}{(x-1)(x+1)^6}+\frac{189}{(x-1)^2(x+1)^5}+\frac{175}{(x-1)^3(x+1)^4}+\frac{175}{(x-1)^4(x+1)^3}+\frac{189}{(x-1)^5(x+1)^2}+\frac{231}{(x-1)^6(x+1)}+\frac{429}{(x-1)^7}\right)}{128\sqrt{(x-1)(x+1)}}$ |
| 9 | $\frac{315\left(\frac{6435}{(x+1)^8}+\frac{3432}{(x-1)(x+1)^7}+\frac{2772}{(x-1)^2(x+1)^6}+\frac{2520}{(x-1)^3(x+1)^5}+\frac{2450}{(x-1)^4(x+1)^4}+\frac{2520}{(x-1)^5(x+1)^3}+\frac{2772}{(x-1)^6(x+1)^2}+\frac{3432}{(x-1)^7(x+1)}+\frac{6435}{(x-1)^8}\right)}{256\sqrt{(x-1)(x+1)}}$ |

Table 2.2: Derivatives of arcosh(x) up to the 9-th order

**Theorem 2.3.1.** *The n-th derivative of* acosh *reads:*

$$\operatorname{acosh}^{(n)}(x)=\frac{(-1)^{(n+1)}}{2^{(n-1)}(x^2-1)^{\frac{1}{2}}}\sum_{i=0}^{\lfloor\frac{(n-1)}{2}\rfloor}\frac{(2n-3-2i)!!\mathscr{T}^i(n-i)(2i-1)!!}{2^{\delta(n-2i-1)}}\left(\frac{1}{(x+1)^{n-i-1}(x-1)^i}+\frac{1}{(x+1)^i(x-1)^{n-i-1}}\right) \tag{2.12}$$

*where $\delta(x)$ is the Kronecker delta.*

*Proof.* As before we define

$$\alpha_i^n := (2n-3-2i)!!\mathscr{T}^i(n-i)(2i-1)!!$$

Then we proceed as before by induction. First, we can trivially see that the formula is correct for a starting $n_0$ case. Then we will prove that if the formula is valid for the $n$-th order derivative, it is also

valid for the $(n+1)$-th order derivative.

Computing the derivative of the formula above we see indeed that

$$\begin{aligned}
&\frac{\partial}{\partial x}\left(\frac{1}{(x+1)^{n-i-\frac{1}{2}}(x-1)^{i+\frac{1}{2}}}+\frac{1}{(x+1)^{i+\frac{1}{2}}(x-1)^{n-i-\frac{1}{2}}}\right)\\
&=\left(\frac{1}{2}+i-n\right)\left(\frac{1}{(x-1)^{n-i+\frac{1}{2}}(x+1)^{i+\frac{1}{2}}}+\frac{1}{(x+1)^{n-i+\frac{1}{2}}(x-1)^{i+\frac{1}{2}}}\right)\\
&\quad+\left(-i-\frac{1}{2}\right)\left(\frac{1}{(x-1)^{n-i-\frac{1}{2}}(x+1)^{i+\frac{3}{2}}}+\frac{1}{(x+1)^{n-i-\frac{1}{2}}(x-1)^{i+\frac{3}{2}}}\right)
\end{aligned}$$

From this, we notice that the relation we need to check is

$$\alpha_{i+1}^{n+1}=\left(-\frac{1}{2}-i\right)\alpha_i^n+\left(\frac{3}{2}+i-n\right)\alpha_{i+1}^n$$

This is long and tedious but easy and one can proceed as in the proof of atan □

## 2.4 Faddeeva function

The Faddeeva function, denoted as $f(z)$, is a complex function widely used in physics and engineering, particularly in problems involving wave propagation and scattering in complex media. In accelerator beam physics, it is used when dealing with the 'beam-beam' effect: this phenomenon is a nonlinear interaction in particle colliders caused by the electromagnetic fields of colliding charged particle beams [10] [30]. This can lead to tune shifts, resonance-driven instabilities, and phase-space distortions, affecting beam quality and stability. Managing this effect is critical in high-luminosity colliders to ensure efficient operation and maximize collision rates.

The Faddeeva function is defined as:

$$w(z)=e^{-z^2}\left(1+\frac{2i}{\sqrt{\pi}}\int_0^z e^{t^2}\,dt\right),$$

where $z$ is a complex variable. This function has an exponential decay term, $e^{-z^2}$, and an integral that introduces a complex component.

For the Faddeeva function, there is a well-known recursive relation [25] to express and evaluate its

derivatives:

$$w^{(n+2)} = -2zw^{(n+1)} - 2(n+1)w^{(n)} \quad n = 0, 1, \ldots \tag{2.13}$$

The problem with this recursive relation is that it becomes very unstable (for example, at $z = 5$, we lose significant accuracy, and many orders become incorrect; if $z$ is further increased, we observe an increasing discrepancy, with values reaching $10^{100}$ instead of $10^{-50}$). Overall, the recursive relation is accurate as long as we stay inside $A = \{z \in \mathscr{C} \mid |z| < 1\}$, that is, when we deal with inputs with small absolute values.

Therefore, we sought an alternative method for expressing the derivatives of the function to use them inside the Truncated Power Series Algebra. To further analyze $w(z)$ and related functions, we define two sequences of polynomials, $p_n(z)$ and $q_n(z)$, which follow specific recursive rules. These definitions are as follows:

1. The initial values for the sequences are:

$$p_0 = 1, \quad q_1 = 1, \quad p_n = 0 \text{ for } n < 0, \quad \text{and} \quad q_n = 0 \text{ for } n < 1.$$

2. For $n \geq 1$, the polynomials $p_n(z)$ are defined recursively as:

$$p_n = -2z \cdot p_{n-1} + p'_{n-1},$$

where $p'_{n-1}$ denotes the derivative of $p_{n-1}$ with respect to $z$.

3. The polynomials $q_n(z)$ are defined recursively as a linear combination of the $p_k(z)$ polynomials:

$$q_n = \sum_{k=0}^{n-1} c_k \cdot p_k,$$

where each coefficient $c_k$ depends on the previous recursive steps and the properties of $q_n$ and $p_n$.

Now let us define them formally:

**Definition 2.4.1.** *The polynomial $p_n$ is defined as follows:*

$$p_n = \begin{cases} 0 & \text{if } n < 0, \\ 1 & \text{if } n = 0, \\ -2z \cdot p_{n-1} + p'_{n-1} = -2z \cdot p_{n-1} - 2(n-1)p_{n-2} & \text{for } n \geq 1, \end{cases}$$

**Definition 2.4.2.** *The polynomial $q_n$ is defined as follows:*

$$q_n = \begin{cases} 0 & \text{if } n < 1, \\ 1 & \text{if } n = 1, \\ \sum_{i=0}^{n-1} 2^{n-i-1} \frac{i!}{(2i-n+1)!} \cdot p_{2i-n+1} & \text{for } n \geq 2. \end{cases}$$

**Lemma 2.4.3.** *The polynomial $q_n$ satisfies the relation*

$$q_{n+1} = -2zq_{n+1} - 2(n+1)q_n \tag{2.14}$$

*Proof.* This can be easily proven by induction (though tedious). Alternatively (and suggested), one can prove the next theorem deductively and then use it to prove that the polynomial $q_n$ must satisfy this relation. □

**Theorem 2.4.4.** *The solution to the nonlinear recursive relation*

$$w^{(n+2)} = -2zw^{(n+1)} - 2(n+1)w^{(n)} \quad n = 0, 1, \ldots$$

*is*

$$w^{(n)}(z) = p_n(z)w(z) + q_n(z)\frac{2i}{\sqrt{\pi}} \tag{2.15}$$

*Proof.* We will check that this is indeed a solution by induction.

**Base case:** For $n = 1$,

$$w'(z) = -2zw(z) + \frac{2i}{\sqrt{\pi}} = p_1(z)w(z) + q_1(z)\frac{2i}{\sqrt{\pi}}.$$

**Induction case:** Assuming that eq. 2.15 holds for $n+1$, we need to prove that it holds for $n+2$:

$$\begin{aligned} w^{(n+2)} &= -2zw^{(n+1)} - 2(n+1)w^{(n)} \\ &= -2z\left(p_{n+1}(z)w(z) + q_{n+1}(z)\frac{2i}{\sqrt{\pi}}\right) - 2(n+1)\left(p_n(z)w(z) + q_n(z)\frac{2i}{\sqrt{\pi}}\right) \\ &= w(z)\left(-2zp_{n+1}(z) - 2(n+1)p_n(z)\right) + \frac{2i}{\sqrt{\pi}}\left(-2zq_{n+1}(z) - 2(n+1)q_n(z)\right) \\ &= p_{n+2}(z)w(z) + q_{n+2}\frac{2i}{\sqrt{\pi}}, \end{aligned}$$

where we have used the relations $q_{n+2} = -2zq_{n+1} - 2(n+1)q_n$ and $p_{n+2} = -2zp_{n+1} - 2(n+1)p_n$.

□

Equation eq. 2.15 is numerically much more stable compared to previous approaches. However, when tested with very large values, an important difficulty arises. Specifically, if we consider the complex domain $A = \{z = re^{i\theta} \in \mathbb{C} \mid -\frac{\pi}{4} < \theta < \frac{5}{4}\pi\}$, the function and its derivatives decay to zero very quickly as $|z|$ becomes larger. Unfortunately, in our expression for $w^{(n)}(z)$ (Equation eq. 2.15), both terms, $p_n(z)w(z)$ and $q_n\frac{2i}{\sqrt{\pi}}$, become very large when evaluated for large $|z|$, with opposite signs. This results in the subtraction of quantities larger than $10^{100}$, while we expect the final difference to be smaller than $10^{-40}$. Therefore, in the domain $A$, we need to adopt a perturbative approach.

A known asymptotic expansion of the Faddeeva function $w(z)$ at $z \to \infty$ ([25]) can help. It is given by:

$$w(z) \sim \frac{i}{\sqrt{\pi}z} \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2z^2)^n},$$

where $(2n-1)!!$ is the double factorial defined as:

$$(2n-1)!! = (2n-1)(2n-3)\cdots 3 \cdot 1, \quad (0!! = 1).$$

Expanding the series explicitly, we obtain:

$$w(z) \sim \frac{i}{\sqrt{\pi}z}\left(1 + \frac{1}{2z^2} + \frac{3}{(2z^2)^2} + \frac{15}{(2z^2)^3} + \cdots\right).$$

The dominant term as $|z| \to \infty$ is:

$$w(z) \sim \frac{i}{\sqrt{\pi}z}.$$

Thus, we can derive the series to get all the derivatives we need (when performing this, one should always check if the hypotheses to exchange the derivation and the infinite summation hold). Proceeding

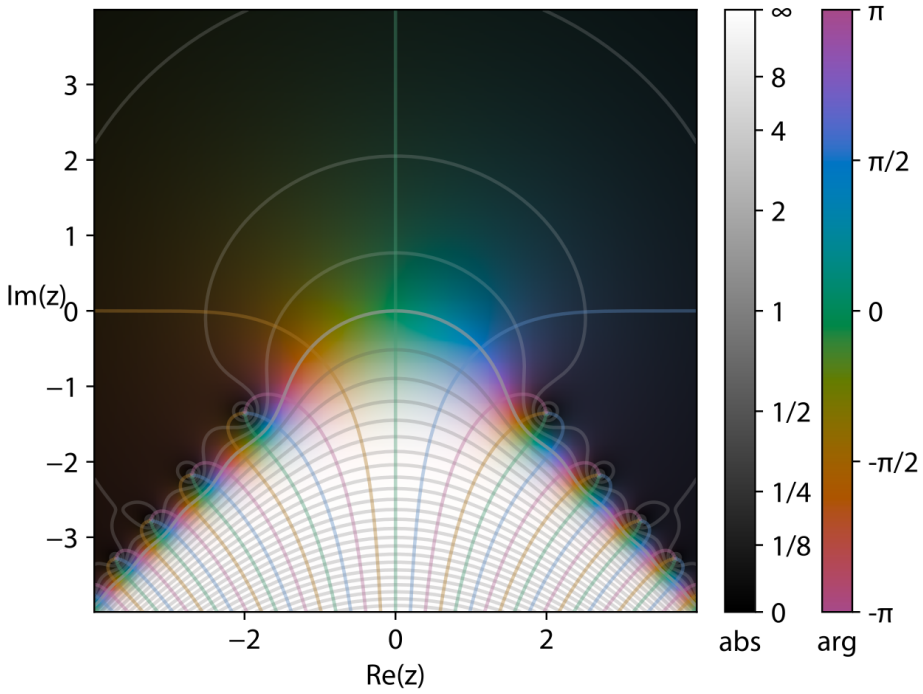



Figure 2.1: Values of the Faddeeva function inside the complex plane (Picture from Wikipedia).

with the derivatives, we get:

$$
\begin{aligned}
w^{(0)}(z) &= \frac{i}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(2n-1)!!}{2^n} z^{-2n}, \\
w^{(1)}(z) &= -\frac{i}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(2n-1)(2n+1)}{2^n} z^{-(2n+2)}, \\
w^{(2)}(z) &= \frac{i}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(2n-1)(2n+1)(2n+2)}{2^n} z^{-(2n+3)}, \\
&\vdots \\
w^{(m)}(z) &= (-1)^m \frac{i}{\sqrt{\pi}} \sum_{n=0}^{\infty} \frac{(2n-1)!!}{2^n} z^{-(2n+m)} \frac{(2n+m)!}{(2n)!}.
\end{aligned}
\tag{2.16}
$$

This approach is very stable and accurate for $|z| > 10$ in the domain $A$ already with only 7 coefficients taken into account in the summations.

## 2.5 Sinc and Sinhc

In this section, we present how we dealt with the instabilities of $\mathrm{sinc}(x) = \frac{\sin(x)}{x}$ and $\mathrm{sinhc}(x) = \frac{\sinh(x)}{x}$. In this case, the instabilities are generated by the division by $x$ when we get near 0. This problem can be solved by a perturbative approach since it is related to only a single point where the functions are differentiable an infinite number of times.

Our strategy is as follows: we expand the functions and their derivatives near 0 and then follow the same procedure as in Equation eq. 2.16. As for the Faddeeva function expansion, we notice that to obtain continuity modulo $1 \times 10^{-16}$ (i.e. errors smaller than the machine precision of double floating point numbers ($\approx 2.22 \times 10^{-16}$)), it is more than sufficient to store the first 7 coefficients of the expansions (which corresponds to going up to the 13th order in the following expansions) in the domain of this perturbative approach. For both functions, the domain is:

$$A = \{z \in \mathbb{C} \mid |z| < 0.75\}.$$

It is worth mentioning that the procedure used to obtain the following results is based on the uniform convergence of the series inside the domain $A$ (for example, one can use Weierstrass criterion). The strategy is to derive term by term the series of sinc computed at $x = 0$ to obtain other series expansions for the derivatives at $x = 0$.

Thus, for sinc, we have:

$$\begin{aligned}
\mathrm{sinc}^{(0)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n+1)!}, \\
\mathrm{sinc}^{(1)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n-1}}{(2n+1)!} 2n, \\
\mathrm{sinc}^{(2)}(x) &= \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n-2}}{(2n+1)!} 2n(2n-1), \\
&\vdots \\
\mathrm{sinc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} (-1)^n \frac{x^{2n-m}}{(2n+1)!} \frac{(2n)!}{(2n-m)!} \\
&= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} (-1)^n \frac{x^{2n-m}}{(2n+1)(2n-m)!}.
\end{aligned} \tag{2.17}$$

Something similar holds for sinhc:

$$\begin{aligned}
\operatorname{sinhc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n+1)!}, \\
\operatorname{sinhc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n-1}}{(2n+1)!} 2n, \\
\operatorname{sinhc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{x^{2n-2}}{(2n+1)!} 2n(2n-1), \\
&\vdots \\
\operatorname{sinhc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{x^{2n-m}}{(2n+1)!} \frac{(2n)!}{(2n-m)!} \\
&= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{x^{2n-m}}{(2n+1)(2n-m)!}.
\end{aligned} \tag{2.18}$$

This approach is very stable and computationally efficient: indeed, we can see that the terms of the summation are rapidly decreasing, allowing us to consider only a limited number of terms in the series.

## 2.6 Asinc and Asinhc

In this section, we explore a method to avoid the instabilities for $\operatorname{asinc}(x) = \frac{\arcsin(x)}{x}$ and $\operatorname{arcsinhc}(x) = \frac{\operatorname{arcsinh}(x)}{x}$. The instabilities occur due to the division by $x$ near zero. We proceed in a similar manner to $\operatorname{sinc}(x)$ and $\operatorname{sinhc}(x)$ (note that $\operatorname{asinhc}(x)$ has an additional source of instability due to the singularity of the derivatives at $x = 1$). As before, we first compute the expansion around 0 and then compute the derivatives, always exploiting the uniform convergence of the series.

$$
\begin{aligned}
\operatorname{asinc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n}}{2n+1} \\
\operatorname{asinc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n-1}}{2n+1} 2n \\
\operatorname{asinc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n-2}}{2n+1} 2n(2n-1) \\
&\vdots \\
\operatorname{asinc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(2n-1)!!}{(2n)!!} \frac{x^{2n-m}}{2n+1} \frac{(2n)!}{(2n-m)!} \\
&= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{((2n-1)!!)^2}{(2n-m)!} \frac{x^{2n-m}}{2n+1}
\end{aligned}
\tag{2.19}
$$

With the same approach, we obtain the derivatives of $\operatorname{asinhc}(x)$:

$$
\begin{aligned}
\operatorname{asinhc}^{(0)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n(2n-1)!!}{(2n)!!} \frac{x^{2n}}{2n+1} \\
\operatorname{asinhc}^{(1)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n(2n-1)!!}{(2n)!!} \frac{x^{2n-1}}{2n+1} 2n \\
\operatorname{asinhc}^{(2)}(x) &= \sum_{n=0}^{\infty} \frac{(-1)^n(2n-1)!!}{(2n)!!} \frac{x^{2n-2}}{2n+1} 2n(2n-1) \\
&\vdots \\
\operatorname{asinhc}^{(m)}(x) &= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(-1)^n(2n-1)!!}{(2n)!!} \frac{x^{2n-m}}{2n+1} \frac{(2n)!}{(2n-m)!} \\
&= \sum_{n=\lceil \frac{m}{2} \rceil}^{\infty} \frac{(-1)^n((2n-1)!!)^2}{(2n-m)!} \frac{x^{2n-m}}{2n+1}
\end{aligned}
\tag{2.20}
$$

In this case, we observe that the terms of the summation decrease as $n$ increases, but at a slower rate compared to the previous case of $\operatorname{sinc}(x)$ and $\operatorname{sinhc}(x)$. Thus, we will check experimentally the threshold of convergence to achieve the best accuracy. Once determined, we will retain the appropriate number of terms (alternatively, one could manually estimate the error of the remainder of the summation as we did for the previous case of $\operatorname{sinc}(x)$ and $\operatorname{sinhc}(x)$).

# Chapter 3

# Numerical Results

In this chapter, we discuss the implementation of the newly derived series and the numerical results obtained using them. These results are compared with those generated by previously established methods. For this analysis, we utilize software developed at CERN called *MAD-NG*[5].

*MAD-NG* (Methodical Accelerator Design—Next Generation) is an advanced computational tool used for designing and simulating particle accelerators. It is specifically tailored for high-performance optics computing, offering improved scalability and precision compared to its predecessors. The software incorporates cutting-edge algorithms for beam dynamics and nonlinear optics, making it an essential tool for accelerator physicists. More information about *MAD-NG* can be found at the following link: https://mad.web.cern.ch/mad/

Specifically, *MAD-NG* provides the framework for implementing the new series and comparing them with the previous standard implementations. As a first step, we will discuss the data generation procedure used to create the testing dataset. Following this, we will analyze the performance of the functions, focusing on the regions where the standard approach exhibited issues with accuracy and stability.

In *MAD-NG*, prior to the advancements presented in this work, the functions atan, acot, atanh, acoth, and acosh operated reliably up to order 6 using manually expanded stable formulas. For higher orders, standard series expansions were employed, but these introduced minor inaccuracies comparable to those observed in PTC/FPP. Conversely, for sinc, sinhc, asinc, asinhc, and the Faddeeva function, the advancements presented in this work enhanced the stability even for lower orders.

## 3.1 Data Generation and Testing Framework

### 3.1.1 Data Generation

We first focus on the procedure we followed to generate the data. It is essential to rely on software that provides sufficient precision. Indeed, it is important to remember that within our framework, all calculations should have a relative accuracy close to the value $10^{-16}$, as we are not performing any approximations (except when dealing with perturbative approaches). This requirement is the reason why the data generation framework is based on symbolic calculation software. Specifically, we used Wolfram Mathematica 14.1 and the Python package 'sympy'. The choice of using two packages stems from the fact that we treat these tools as black boxes, and we cannot verify if their answers are correct unless we have other means of comparison. For this reason, it is crucial to double-check the data generated, especially since, in the cases analyzed in this work, both Mathematica and Sympy face various challenges due to the unstable nature of the problems we are solving (e.g., both packages exhibit severe instabilities when dealing with the Faddeeva function, $\mathrm{sinc}(x)$, $\mathrm{sinhc}(x)$, etc.).

In such cases, we needed to rely on external methods (e.g., perturbative analyses) to understand the correct behavior of the functions. This helped us later find a way to make both packages work correctly. For example, using the standard definitions of the Faddeeva function leads to severe numerical issues in our range of use; thus, we had to rephrase the expression to be evaluated, eventually obtaining the correct result. The overall procedure can be summarized as follows:

1. Generation of the data using Sympy;
2. Generation of the data using Mathematica;
3. Comparison of the data generated through the two different sources;
4. Testing of the functions with the verified data.

Finally, it is worth noting that the data is generated with a precision of 25 significant digits. This precision is sufficient for testing the validity of our approach, as our limit is the machine precision of double floating point numbers $\approx 2.22 \times 10^{-16}$, which corresponds to verifying up to 16 significant digits at most.

We will now describe the method we use to evaluate the performance of the new functions.

### 3.1.2 Performance Analysis

To test the performance of our functions, we sample points in the domain

$$A = \{z \in \mathbb{C} \mid |z| < 10\}$$

as well as points with large absolute values up to $10^8$. Specifically, we emphasize very small values down to $x_0 = 10^{-8}$, values in the neighborhood of 1 (as 1 is a transition point for many functions where their behavior changes), points in the vicinity of $i$, and specific multiples of $\pi$ (as they generate alternating signs in some series). Additionally, we verify if the function remains stable for large values. After this general analysis, we inspect points specific to the functions under consideration, such as singularities, boundaries where their behavior changes, and so on.

To measure performance, we use an adaptive metric used in *MAD-NG* and defined as

$$\|x - y\| = \frac{|x - y|}{|\max(x, 1)|},$$

which is essentially an adaptive relative error designed to handle very large and small numbers simultaneously. This approach is useful because, for example, if we subtract $x = 10^{-30}$ and $y = 10^{-31}$, the relative error would indicate 90%, but the values are so small that this inaccuracy is insignificant.

After defining the norm, we proceed describing the procedure we follow:

- We create a TPSA object $P$, setting the evaluation point to $x_0$ and the differential unit to 1 (this means that in our dual algebra, we have the point $x = x_0 + \varepsilon$);
- We take the function $f$ we want to test and we compute $f(P)$;
- We create another TPSA object $Q$ filled with all the correct coefficients computed using Sympy. This corresponds to creating the dual number

  $$x = \sum_{i=0}^{N} \frac{f^{(i)}(x_0)}{i!} \varepsilon^i,$$

  where $N$ is the maximum order of the differential unit, typically set to $N = 15$);
- We compute $d = \|P - Q\|$ using the previously defined metric, applying it component-wise;
- We take the maximum of the computed distance, $\|d\|_\infty$, and represent it as a function of the machine precision eps, i.e.,

  $$\frac{\|d\|_\infty}{\text{eps}},$$

where eps $\approx 2.22 \times 10^{-16}$ for double floating point precision as previously defined.

In order to better visualize the error and observe in which order the values start to become unstable, we will present the error for each order (not only the largest one). Moreover, the errors will be shown as a function of the machine precision unit 'eps', using a cutoff value. Specifically, we use

$$\max\left(\frac{\|P-Q\|}{\text{eps}}, 1\right)$$

to remain consistent with the metrics defined above.

Now, we proceed to describe the implementation of the mathematical formulas introduced in Chapter 2.

## 3.2 Numerical Performance

In this section, we analyze the numerical implementation of the formulas described in Chapter 2 and demonstrate the main improvements compared to previous implementations. The following subsections are organized as follows:

- Comparison of the results at specific points where the previous implementation is unstable;
- Analysis of the results and improvements.

### 3.2.1 Atan

We now analyze the performance of Atan. (The C code can be found in the appendix along with a brief description focusing on the mathematical definition in Section B.1.) In the following discussion, we compare the previous approach with the new one introduced in Chapter 2. For the function under consideration, we evaluate both the previous implementation (referred to as 'OLD_SERIES' in the code) and the new approach. Specifically, we compare the numerical accuracy and stability of

$$\text{atan}(x) = -\frac{i}{2}\ln\left(\frac{i-x}{i+x}\right)$$

with Equation eq. 2.3.

We now illustrate the difference in performance at specific representative points between the previous and the new approach. The errors are presented as multiples of the machine precision unit

'eps' for each order. On the left, we show the errors obtained with the previous approach, while on the right, we display the errors from the new approach. Specifically, our analysis will be focused around 0 given that this is the point where we were facing an unstable behaviour of the previous implementation. Then we will also analyze the behaviour around the singularities.

### Comparison at $x_0 = 0.1$

**Point:** $x_0 = 0.1$

| | **Previous approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.8750000000000000 \times 10^{0}$ | 5 |
| 7 | $5.9375000000000000 \times 10^{0}$ | 6 |
| 8 | $3.1812500000000000 \times 10^{+1}$ | 7 |
| 9 | $2.7562500000000000 \times 10^{+1}$ | 8 |
| 10 | $1.7162500000000000 \times 10^{+2}$ | 9 |
| 11 | $2.2687500000000000 \times 10^{+2}$ | 10 |
| 12 | $1.7715937500000000 \times 10^{+3}$ | 11 |
| 13 | $1.3632500000000000 \times 10^{+3}$ | 12 |
| 14 | $8.4610156250000000 \times 10^{+3}$ | 13 |
| 15 | $3.0157875000000000 \times 10^{+4}$ | 14 |
| 16 | $8.4493683593750000 \times 10^{+4}$ | 15 |

Maximum Error: 84494 eps, at $x_0 = 0.1$ observed at order 15 .

| | **New approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, at $x_0 = 0.1$ observed at order 3 .

Here we check the stability of the function when we are a near to 0 but not too much. The results shows that the new approach has an impressive accuracy, especially if compared to the previous approach. Moreover, going towards zero, the instability of the old approach further increases as we will see.

## Comparison at $x_0 = \frac{\pi}{3}i$

**Point:** $x_0 = \frac{\pi}{3}i$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.4148475504056880 \times 10^{16}$ | 0 |
| 2 | $1.1594725347858086 \times 10^{1}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $3.2298691768100241 \times 10^{1}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $5.9553733096040816 \times 10^{1}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $9.5092423640831768 \times 10^{1}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.5813216984796364 \times 10^{1}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $3.5620189341245663 \times 10^{2}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $4.9228172283738519 \times 10^{2}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.8999872411265883 \times 10^{3}$ | 15 |

Maximum Error:
$1.4148475504057 \times 10^{16}$ eps,
observed at order 0 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.1594725347858086 \times 10^{1}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $3.3590639438824240 \times 10^{1}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $5.5869997028244526 \times 10^{1}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $7.7446406882739581 \times 10^{1}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $9.9910780040302441 \times 10^{1}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.2173988762196593 \times 10^{2}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.4383540049982327 \times 10^{2}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.6440488288519845 \times 10^{2}$ | 15 |

Maximum Error: 165 eps,
observed at order 15 .

$x_0 = \frac{\pi}{3}i$ is a point near to the singularity in the complex plane. Here we notice that the scalar part of the previous approach is totally off and moreover the coefficients are getting less and less accurate as we increase the order. We notice that even in this case the new approach is improving the accuracy but it is also presenting inaccuracies as well for high orders (due to the singularity $x = i$).

**Comparison at $x_0 = 0.1i$**

**Point:** $x_0 = 0.1i$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $3.3750000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $5.7250000000000000 \times 10^{1}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.4725000000000000 \times 10^{2}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $2.7786250000000000 \times 10^{3}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $3.8330625000000000 \times 10^{4}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0859900000000000 \times 10^{5}$ | 15 |

Maximum Error: 108599 eps, observed at order 15 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 14 .

Here we check $x_0 = 0.1i$ to verify the stability around 0 when we come from the imaginary axis. Even in this case, we see that the new approach is outperforming the old one.

**Comparison at $x_0 = 10^{-6}$**

**Point:** $x_0 = 10^{-6}$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $3.6250000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $4.4125000000000000 \times 10^{1}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $2.0250000000000000 \times 10^{1}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $2.4707500000000000 \times 10^{3}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $5.5160625000000000 \times 10^{3}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $3.1138025000000000 \times 10^{5}$ | 15 |

Maximum Error: 311381 eps, observed at order 15 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 1 .

Here we see that even if we are very near to 0 the new approach is still incredibly accurate and it is outperforming the old one.
Overall, the new approach is much more stable and efficient, especially in the domain $\{z \in \mathbb{C} \mid \|z| < 1.1\}$.

### 3.2.2 Acot, Atanh, and Acoth

We do not present the code used for the implementation of acot, atanh, and acoth since it is very similar to the one used for atan. The improvements are also quite similar to those achieved with the new implementation of atan. Specifically, all the new functions demonstrate much better accuracy in the domain

$$\{z \in \mathbb{C} \mid |z| < 1.1\}$$

as well as around singularities (around the singularities, the new formulas reduce the error but still exhibit some inaccuracies). For this reason, we omit the discussion.

It is worth noting that the previous implementation of acot relied on the equivalence:

$$\operatorname{acot}(x) = \frac{i}{2}\log\left(\frac{x-i}{x+i}\right),$$

whereas for atanh and acoth, the formulas used are the standard ones for the real domain:

$$\operatorname{atanh}(x) = \frac{1}{2}\log\left(\frac{1+x}{1-x}\right) \qquad \|x\| \leq 1,$$

$$\operatorname{acoth}(x) = \frac{1}{2}\log\left(\frac{1+x}{1-x}\right) \qquad \|x\| \geq 1.$$

Overall, the new implementations significantly improve accuracy and stability, particularly around 0, as observed in the case of atan.

### 3.2.3 Acosh

In this section, we test the performance of acosh. We compare the new formula (Equation eq. 2.12) with the old implementation based on the identity:

$$\operatorname{acosh}(x) = \ln(x + \sqrt{x^2 - 1}).$$

The instability observed for acosh arises from the singularity of its derivatives around $x = 1$. Therefore, we focus specifically on this point. The corresponding code can be found in Appendix B.5.

In this case, the new approach aims to address the instabilities caused by the singularity at $x = 1$. Overall, the new method improves upon the previous approach, but the instability near the singularity remains.

**Comparison at $x_0 = 1.000001$**

**Point:** $x_0 = 1.000001$

| | Previous Approach | | New Approach | | |
|---|---|---|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** | **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.2069042968750000 \times 10^{2}$ | 0 | 1 | $2.3764583488195634 \times 10^{0}$ | 0 |
| 2 | $8.5155137320644790 \times 10^{4}$ | 1 | 2 | $7.7244657686125083 \times 10^{2}$ | 1 |
| 3 | $2.5546406642617457 \times 10^{5}$ | 2 | 3 | $2.3173403424030284 \times 10^{3}$ | 2 |
| 4 | $4.2577345718131086 \times 10^{5}$ | 3 | 4 | $8.4020411905578985 \times 10^{3}$ | 3 |
| 5 | $5.9608299379351037 \times 10^{5}$ | 4 | 5 | $1.1762854322072972 \times 10^{4}$ | 4 |
| 6 | $7.6639147943352477 \times 10^{5}$ | 5 | 6 | $1.5123684769709163 \times 10^{4}$ | 5 |
| 7 | $9.3670105141980434 \times 10^{5}$ | 6 | 7 | $1.8484500634255277 \times 10^{4}$ | 6 |
| 8 | $1.1070112260683978 \times 10^{6}$ | 7 | 8 | $2.1845308833148971 \times 10^{4}$ | 7 |
| 9 | $1.2773205637951044 \times 10^{6}$ | 8 | 9 | $2.5206135995561948 \times 10^{4}$ | 8 |
| 10 | $1.4476289338701826 \times 10^{6}$ | 9 | 10 | $2.8566951571758367 \times 10^{4}$ | 9 |
| 11 | $1.6179393993359790 \times 10^{6}$ | 10 | 11 | $3.1927770091780083 \times 10^{4}$ | 10 |
| 12 | $1.7882479277186738 \times 10^{6}$ | 11 | 12 | $3.5288575448213080 \times 10^{4}$ | 11 |
| 13 | $1.9585590862990867 \times 10^{6}$ | 12 | 13 | $3.8649404546716665 \times 10^{4}$ | 12 |
| 14 | $2.1288677870886698 \times 10^{6}$ | 13 | 14 | $4.2010227757040942 \times 10^{4}$ | 13 |
| 15 | $2.2991750868642027 \times 10^{6}$ | 14 | 15 | $4.5371035168825969 \times 10^{4}$ | 14 |
| 16 | $2.4694843182200179 \times 10^{6}$ | 15 | 16 | $4.8731851282699979 \times 10^{4}$ | 15 |
| | Maximum Error: $2.4694843182200179 \times 10^{6}$ eps, observed at order 15 . | | | Maximum Error: $4.8731851282699979 \times 10^{4}$ eps, observed at order 15 . | |

As we can see, the new approach improves accuracy (by a factor of 100) but still exhibits problems due to the singularity of the derivatives. Outside the neighborhood of 1, both approaches perform very well, consistently achieving machine precision accuracy.

### 3.2.4 Faddeeva Function

Here we present the performance analysis of the Faddeeva function. Specifically, we compare the new formula obtained in Equation eq. 2.15 with the old implementation based on:

$$w(z) = \mathrm{erfcx}(-iz), \quad \mathrm{erfcx}(z) = e^{-z^2}\left(1 - \mathrm{erf}(-iz)\right).$$

and the stable formula derived for the error function as part of the GTPSA framework.

To account for the different behaviors of the function, as discussed in the theoretical developments section on the Faddeeva function, we implemented three branches in the code to cover all possible scenarios in the complex domain. The corresponding code and its brief mathematical explanation are provided in Appendix B.6.

One final point to address is that, as noted in Equation eq. 2.13, there is an already well-known relation for the derivatives of the Faddeeva function. However, we emphasize that the formula in Equation eq. 2.13 is less stable than our approach.

### Comparison at $x_0 = 1$

**Point:** $x_0 = 1$

| | **Previous Approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $4.5037732096678219 \times 10^{14}$ | 1 |
| 3 | $4.4532746461811550 \times 10^{15}$ | 2 |
| 4 | $5.6139482103745156 \times 10^{14}$ | 3 |
| 5 | $4.4622786780737570 \times 10^{15}$ | 4 |
| 6 | $5.8348081431391738 \times 10^{14}$ | 5 |
| 7 | $4.4638480761692070 \times 10^{15}$ | 6 |
| 8 | $5.8917354889630300 \times 10^{14}$ | 7 |
| 9 | $4.4642737891667770 \times 10^{15}$ | 8 |
| 10 | $5.9096169669658250 \times 10^{14}$ | 9 |
| 11 | $4.4644155285850945 \times 10^{15}$ | 10 |
| 12 | $5.9160650432015550 \times 10^{14}$ | 11 |
| 13 | $4.4644691621280820 \times 10^{15}$ | 12 |
| 14 | $5.9186362426286275 \times 10^{14}$ | 13 |
| 15 | $4.4644913884787495 \times 10^{15}$ | 14 |
| 16 | $5.9197433000164788 \times 10^{14}$ | 15 |

Maximum Error: $4.4644913884787495 \times 10^{15}$ eps, observed at order 14

| | **New Approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 3 .

Here we check $x_0 = 1$ in order to test the first branch of the code shown in B.6, where we use the formula derived and proved in eq. 2.15. As we can notice, it is evident that the new approach is much more stable.

**Comparison at $x_0 = 7.01$**

**Point:** $x_0 = 7.01$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $6.8815885555358160 \times 10^{15}$ | 1 |
| 3 | $5.1591594667756550 \times 10^{14}$ | 2 |
| 4 | $4.5517434546355230 \times 10^{15}$ | 3 |
| 5 | $3.8357952196568681 \times 10^{14}$ | 4 |
| 6 | $4.4765561162969125 \times 10^{15}$ | 5 |
| 7 | $4.1469840325789381 \times 10^{14}$ | 6 |
| 8 | $4.4803764120651815 \times 10^{15}$ | 7 |
| 9 | $4.3117511363482631 \times 10^{14}$ | 8 |
| 10 | $4.4813883397505280 \times 10^{15}$ | 9 |
| 11 | $4.3688034614841488 \times 10^{14}$ | 10 |
| 12 | $4.4817336255584970 \times 10^{15}$ | 11 |
| 13 | $4.3904796101476856 \times 10^{14}$ | 12 |
| 14 | $4.4818701826899785 \times 10^{15}$ | 13 |
| 15 | $4.3994745862836444 \times 10^{14}$ | 14 |
| 16 | $4.4819288125781150 \times 10^{15}$ | 15 |

Maximum Error: $6.8815885555358 \times 10^{15}$ eps, observed at order 1.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 4.

Here we check the accuracy of the new method near to the point where we switch to the perturbative approach (for points with absolute values larger than 7 and such that they respect the inequality $\mathrm{Im}(z) >= -|\mathrm{Re}(z)|$). We check at the border because the perturbative approach we use is an expansion at $z = \infty$, so we expect the minimum accuracy to be at the 'furthest' point from $z = \infty$. As we can see the perturbative approach is already performing really well. Moreover, we tested as well the first branch, i.e. the one implementing eq. 2.15, when we are very near to 7, before switching to the perturbative approach, obtaining good accuracy. This ensures that the error is not exploding going from one branch to the other.

**Comparison at $x_0 = 100$**

**Point:** $x_0 = 100$

| Previous Approach | | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $4.5035996442635000 \times 10^{13}$ | 1 |
| 3 | $4.5030930072371470 \times 10^{15}$ | 2 |
| 4 | $5.6293412323324141 \times 10^{13}$ | 3 |
| 5 | $4.5031848984217345 \times 10^{15}$ | 4 |
| 6 | $5.8544775615491875 \times 10^{13}$ | 5 |
| 7 | $4.5032008586995415 \times 10^{15}$ | 6 |
| 8 | $5.9125765535819336 \times 10^{13}$ | 7 |
| 9 | $4.5032051851921425 \times 10^{15}$ | 8 |
| 10 | $5.9308320363891961 \times 10^{13}$ | 9 |
| 11 | $4.5032066254084640 \times 10^{15}$ | 10 |
| 12 | $5.9374156837077625 \times 10^{13}$ | 11 |
| 13 | $4.5032071703442730 \times 10^{15}$ | 12 |
| 14 | $5.9400410470472359 \times 10^{13}$ | 13 |
| 15 | $4.5032073961661765 \times 10^{15}$ | 14 |
| 16 | $5.9411714436177109 \times 10^{13}$ | 15 |

Maximum Error: $4.5032073961661765 \times 10^{15}$ eps, observed at order 14 .

| New Approach | | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 1 .

Here we check the accuracy for $z = 100$ because we are switching from the second branch of the function to the third one (B.6): here we use indeed a simplified version of the perturbative approach where we simply use the approximation

$$w(z) = \frac{i}{\sqrt{\pi} z}$$

and thus

$$w^{(n)}(z) = (-1)^{n+1} \frac{i}{\sqrt{\pi} z^n} \qquad n = 1, 2, \ldots$$

This new approach is basically cutting off all the higher-order contributions. We are checking the validity of the approach near the border where we switch from one branch to another because this is the region where the previous method is achieving the worst performance. Yet, the accuracy is perfect and we can see an impressive improvement concerning the previous approach.

**Comparison at $x_0 = 10^{-6}$**

**Point:** $x_0 = 10^{-6}$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.9170735247653172 \times 10^{10}$ | 1 |
| 3 | $5.7067278326545550 \times 10^{15}$ | 2 |
| 4 | $1.0083194689236149 \times 10^{10}$ | 3 |
| 5 | $4.5543096909906270 \times 10^{15}$ | 4 |
| 6 | $9.0037640752461815 \times 10^{9}$ | 5 |
| 7 | $4.5045411221068900 \times 10^{15}$ | 6 |
| 8 | $8.9115142025309219 \times 10^{9}$ | 7 |
| 9 | $4.5036097222366470 \times 10^{15}$ | 8 |
| 10 | $8.8904891547323952 \times 10^{9}$ | 9 |
| 11 | $4.5035996984701370 \times 10^{15}$ | 10 |
| 12 | $8.8831016123221226 \times 10^{9}$ | 11 |
| 13 | $4.5035996277174280 \times 10^{15}$ | 12 |
| 14 | $8.8801651707066174 \times 10^{9}$ | 13 |
| 15 | $4.5035996273630805 \times 10^{15}$ | 14 |
| 16 | $8.8789022410361176 \times 10^{9}$ | 15 |

Maximum Error: $5.7067278326545550 \times 10^{15}$ eps, observed at order 2 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 1 .

Here we check the accuracy of the function near to 0 because we know that the region near to 0 is the one where the function is changing the most and the one in which it is easier to face numerical issues for the scalar part. As we can see from the table above the new implementation is behaving really good also in this case.

**Comparison at $x_0 = -10i$**

**Point:** $x_0 = -10i$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.5537418714428211 \times 10^{17}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $4.1143740543120064 \times 10^{17}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $3.1577171737004077 \times 10^{17}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0558108753203880 \times 10^{17}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $2.1626346290848756 \times 10^{16}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $6.2201833421438140 \times 10^{15}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $4.6145211288905130 \times 10^{15}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error:
$4.114374054312 \times 10^{17}$ eps,
observed at order 4 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed
at order 8 .

Here we check a negative imaginary number $x_0 = -10i$ because in this region of the complex plane, the function is growing exponentially and it is changing behavior (see figure 2.1). In this case eq. 2.15 is working really well without facing any numerical issue (it is expected since the numerical issue was due to the subtraction of two huge numbers with a very small difference whereas in this case, this summation is exploding). Even in this case, the new implementation is really accurate.

**Comparison at $x_0 = 10^7$**

**Point:** $x_0 = 10^7$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $4.5035996273704958 \times 10^{9}$ | 1 |
| 3 | $4.5035996273654295 \times 10^{15}$ | 2 |
| 4 | $5.6294995342115374 \times 10^{9}$ | 3 |
| 5 | $4.5035996273663485 \times 10^{15}$ | 4 |
| 6 | $5.8546795155796251 \times 10^{9}$ | 5 |
| 7 | $4.5035996273665085 \times 10^{15}$ | 6 |
| 8 | $5.9127904785133171 \times 10^{9}$ | 7 |
| 9 | $4.5035996273665515 \times 10^{15}$ | 8 |
| 10 | $5.9310497835747757 \times 10^{9}$ | 9 |
| 11 | $4.5035996273665660 \times 10^{15}$ | 10 |
| 12 | $5.9376348165198689 \times 10^{9}$ | 11 |
| 13 | $4.5035996273665715 \times 10^{15}$ | 12 |
| 14 | $5.9402607334679136 \times 10^{9}$ | 13 |
| 15 | $4.5035996273665735 \times 10^{15}$ | 14 |
| 16 | $5.9413913685908995 \times 10^{9}$ | 15 |

Maximum Error: $4.5035996273665735 \times 10^{15}$ eps, observed at order 14 .

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 0 eps, observed at order 1 .

In this case, we check the accuracy for a very large value because this is usually the magnitude of the input of the function when we deal with the simulation of an elliptic beam in the beam-beam effect in Chapter 4. As we can expect, the accuracy is really good and this is because with very large values our simplified perturbative formula is 'nearer' to the point of expansion $z = \infty$.

Other than that, the function has been tested in many more points, focusing especially on the borders of the various domains related to the three branches (an improvement to be done is to refine the method around the $1D$ domain defined by $\mathrm{Im}(z) = -|\mathrm{Re}(z)|$ where we lose some accuracy). Overall we can say that the new method is really precise and stable and it is largely outperforming the previous one.

### 3.2.5 Sinc

We now analyze the new implementation of the sinc function based on the perturbative approach shown in Equation eq. 2.17. The main issue arises at $x_0 = 0$, where instability is caused by the division. To address this, a perturbative approach around 0, as shown in the previous chapter, is highly effective. While perturbative approaches sacrifice analytical precision, they achieve machine precision accuracy, which in the context of floating-point arithmetic is equivalent to an analytical estimation. The implementation of the code, along with a brief algorithmic explanation, is provided in Appendix B.7.

The code operates as follows:

1. (**First Branch**) For an input such that $|a_0| \leq 1 \times 10^{-12}$, we use the standard Taylor expansion of sinc at $x = 0$:
$$\text{sinc}(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n+1)!}.$$
2. (**Second Branch**) For an input $1 \times 10^{-12} < |a_0| \leq 0.75$, we use the perturbative scheme defined in Equation eq. 2.17.
3. (**Third Branch**) For an input $|a_0| > 0.75$, we use the standard definition of the function $\frac{\sin(x)}{x}$.

The old approach, on the other hand, operated as follows:

1. For an input such that $|a_0| \leq 1 \times 10^{-12}$, we used the Taylor expansion of sinc at $x = 0$:
$$\text{sinc}(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n+1)!}.$$
2. For an input $|a_0| > 1 \times 10^{-12}$, we used the standard definition of sinc.

It is worth noting that we chose the threshold of $x = 0.75$ to ensure the continuity of the error, i.e. when switching from the second branch to the third, the error remains consistent in its order of magnitude.

We now analyze the performance, focusing naturally around 0.

**Comparison at $x_0 = 10^{-6}$**

**Point:** $x_0 = 10^{-6}$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $4.0981689453125000 \times 10^{2}$ | 1 |
| 3 | $8.5653250000000000 \times 10^{4}$ | 2 |
| 4 | $2.0388565240985107 \times 10^{8}$ | 3 |
| 5 | $2.1989702951932031 \times 10^{11}$ | 4 |
| 6 | $3.7529289286551281 \times 10^{14}$ | 5 |
| 7 | $4.5035517681883995 \times 10^{15}$ | 6 |
| 8 | $4.5035996273631785 \times 10^{15}$ | 7 |
| 9 | $4.5035996273703660 \times 10^{15}$ | 8 |
| 10 | $4.5035996273704960 \times 10^{15}$ | 9 |
| 11 | $4.5035996273704960 \times 10^{15}$ | 10 |
| 12 | $4.5035996273704960 \times 10^{15}$ | 11 |
| 13 | $4.5035996273704960 \times 10^{15}$ | 12 |
| 14 | $4.5035996273704960 \times 10^{15}$ | 13 |
| 15 | $4.5035996273704960 \times 10^{15}$ | 14 |
| 16 | $4.5035996273704960 \times 10^{15}$ | 15 |

Maximum Error: $4.5035996273705 \times 10^{15}$ eps, observed at order 9.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 2.

Here we are comparing the two approaches when we are very near to 0: as we can see the previous approach is not working well and the inaccuracies rapidly grow. Instead, the new approach is working really well.

**Comparison at $x_0 = 0.15$**

**Point:** $x_0 = 0.15$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $7.3437500000000000 \times 10^{0}$ | 1 |
| 3 | $2.7875000000000000 \times 10^{1}$ | 2 |
| 4 | $2.8660156250000000 \times 10^{2}$ | 3 |
| 5 | $1.0381796875000000 \times 10^{4}$ | 4 |
| 6 | $4.2675408569335938 \times 10^{4}$ | 5 |
| 7 | $4.2215365405273438 \times 10^{5}$ | 6 |
| 8 | $1.0488130001253128 \times 10^{7}$ | 7 |
| 9 | $1.2908323054248810 \times 10^{7}$ | 8 |
| 10 | $5.3196699921499765 \times 10^{8}$ | 9 |
| 11 | $1.7305328364951632 \times 10^{9}$ | 10 |
| 12 | $2.3683546289916177 \times 10^{9}$ | 11 |
| 13 | $6.5055108938332361 \times 10^{11}$ | 12 |
| 14 | $4.6041114248175205 \times 10^{12}$ | 13 |
| 15 | $1.1059598822756793 \times 10^{13}$ | 14 |
| 16 | $5.9615893769418775 \times 10^{14}$ | 15 |

Maximum Error: $5.9615893769419 \times 10^{14}$ eps, observed at order 15.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 4.

In this second case, we compare the two implementations in the point $x_0 = 0.15$, a bit further from 0 to check the consistency of the perturbative scheme and the performance with respect to the previous one. As one can observe the old implementation is improving for the previous point $x = 0.000001$ but it still presents large inaccuracies whereas the new implementation is achieving a machine precision accuracy.

**Comparison at $x_0 = 0.75$**

**Point:** $x_0 = 0.75$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.1250000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.4843750000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $3.9375000000000000 \times 10^{0}$ | 5 |
| 7 | $3.6298828125000000 \times 10^{0}$ | 6 |
| 8 | $4.5354614257812500 \times 10^{0}$ | 7 |
| 9 | $8.1327590942382812 \times 10^{0}$ | 8 |
| 10 | $8.6326292753219604 \times 10^{0}$ | 9 |
| 11 | $8.2122319638729095 \times 10^{0}$ | 10 |
| 12 | $8.0807279339060187 \times 10^{0}$ | 11 |
| 13 | $1.5716941402642988 \times 10^{1}$ | 12 |
| 14 | $1.0967749557603383 \times 10^{0}$ | 13 |
| 15 | $2.5454915789278402 \times 10^{1}$ | 14 |
| 16 | $6.5027436236090239 \times 10^{1}$ | 15 |

Maximum Error: 66 eps,
observed at order 15.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $2.6250000000000000 \times 10^{0}$ | 1 |
| 3 | $2.6875000000000000 \times 10^{1}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $2.0195312500000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 27 eps,
observed at order 2.

Here we test the performance of the new approach in $x_0 = 0.75$ which is the point where we switch from the second branch to the third one (standard expression of sinc): this is the weakest point of the second branch. We notice that the accuracy of the second branch is getting worse and we start to see numerical inaccuracies as expected. Yet, the inaccuracies are really small and our function is still very accurate.

From the results presented, it is immediately clear that the new approach performs exceptionally well, particularly when compared to the previous one. Moreover, it demonstrates high accuracy even far from 0 (despite being a perturbative approach centered at 0). Additionally, the new approach shows reliable performance in the general tests.

### 3.2.6 Sinhc

For Sinhc, the code and analysis are nearly identical to those presented for sinc (the only difference in the two formulas is a $-$ sign). For this reason, we omit the discussion on Sinhc.

### 3.2.7 Asinc

We now analyze the new implementation of the asinc function, based on the perturbative approach shown in Equation eq. 2.19. The primary issue arises at $x_0 = 0$, where instability is caused by division. A perturbative approach around 0, as discussed in the previous chapter, effectively addresses this issue. As with the other perturbative approaches, while analytical precision is lost, machine precision accuracy can still be achieved. Details of the code implementation, including a brief algorithmic explanation, are provided in Appendix B.9.

The code operates as follows:

1. (**First Branch**) For an input such that $|a_0| \leq 1 \times 10^{-12}$, the Taylor expansion of asinc at $x = 0$ is used:
$$\text{asinc}(x) = \sum_{n=0}^{\infty} \frac{(2n)!}{(2^n n!)^2 (2n+1)} x^{2n}.$$
2. (**Second Branch**) For an input $1 \times 10^{-12} < |a_0| \leq 0.58$, the perturbative scheme defined in Equation eq. 2.19 is applied.
3. (**Third Branch**) For an input $|a_0| > 0.58$, the standard definition of the function $\frac{\arcsin(x)}{x}$ is used.

The old approach, in contrast, operated as follows:

1. For an input such that $|a_0| \leq 1 \times 10^{-12}$, the Taylor expansion of asinc at $x = 0$ was used:
$$\text{asinc}(x) = \sum_{n=0}^{\infty} \frac{(2n)!}{(2^n n!)^2 (2n+1)} x^{2n}.$$
2. For an input $|a_0| > 1 \times 10^{-12}$, the standard definition of asinc was applied.

As in the cases of the sinc and Faddeeva functions, the threshold $x = 0.58$ was chosen to ensure continuity of the error. Specifically, this choice ensures that when transitioning from the second branch to the third branch, the error maintains the same order of magnitude.

We now proceed to analyze the performance, focusing primarily on the behavior around 0.

### Comparison at $x_0 = 10^{-6}$

**Point:** $x_0 = 10^{-6}$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $7.0683000000000000 \times 10^{4}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $4.4562769862916580 \times 10^{15}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $4.5035996273705170 \times 10^{15}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $4.5035996273689839 \times 10^{15}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $4.5035996273703849 \times 10^{15}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $4.5035996273703898 \times 10^{15}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $4.5035996273704789 \times 10^{15}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $4.5035996273704960 \times 10^{15}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: $4.5035996273705 \times 10^{15}$ eps, observed at order 4.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 3.

Here we are comparing the two approaches when we are very near to 0: as we can see the previous approach is not working well and the inaccuracies rapidly grow. Instead, one can notice that the new approach is performing really well.

**Comparison at $x_0 = 0.15$**

**Point:** $x_0 = 0.15$

| | **Previous Approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $7.3092480932094809 \times 10^{0}$ | 1 |
| 3 | $3.3902840932840982 \times 10^{1}$ | 2 |
| 4 | $3.0293840932840980 \times 10^{2}$ | 3 |
| 5 | $2.3409580394805980 \times 10^{4}$ | 4 |
| 6 | $3.3209840932840938 \times 10^{4}$ | 5 |
| 7 | $4.2215365405273438 \times 10^{5}$ | 6 |
| 8 | $1.3947398493849389 \times 10^{7}$ | 7 |
| 9 | $3,2309480923850808 \times 10^{7}$ | 8 |
| 10 | $4.4308309845038508 \times 10^{8}$ | 9 |
| 11 | $1.3984029830588088 \times 10^{9}$ | 10 |
| 12 | $2.3320948029384082 \times 10^{9}$ | 11 |
| 13 | $2.4093850934850983 \times 10^{11}$ | 12 |
| 14 | $3.4309850943850984 \times 10^{12}$ | 13 |
| 15 | $2.3094045894380580 \times 10^{13}$ | 14 |
| 16 | $5.4648509433094845 \times 10^{14}$ | 15 |

Maximum Error: $5.4648509433094845 \times 10^{14}$ eps, observed at order 15.

| | **New Approach** | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $1.0000000000000000 \times 10^{0}$ | 1 |
| 3 | $1.0000000000000000 \times 10^{0}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $1.0000000000000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 1 eps, observed at order 3.

In this second case we compare the two implementations in a point $x_0 = 0.15$, a bit further from 0, to check the consistency of the perturbative scheme and the performance with respect to the previous approach. As one can observe the old implementation is improving for the previous point $x = 0.000001$ but it still presents large inaccuracies whereas the new implementation is achieving a machine precision accuracy.

**Comparison at $x_0 = 0.58$**

**Point:** $x_0 = 0.58$

| | Previous Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $3.6095620848369598 \times 10^{0}$ | 1 |
| 3 | $4.1225973592005415 \times 10^{0}$ | 2 |
| 4 | $2.5937321806349274 \times 10^{1}$ | 3 |
| 5 | $1.5000000000000000 \times 10^{1}$ | 4 |
| 6 | $2.2366795922458699 \times 10^{1}$ | 5 |
| 7 | $1.8331017230369138 \times 10^{1}$ | 6 |
| 8 | $4.7944930492735409 \times 10^{0}$ | 7 |
| 9 | $1.1136944914150037 \times 10^{1}$ | 8 |
| 10 | $1.5921429400796914 \times 10^{1}$ | 9 |
| 11 | $7.4413972989468613 \times 10^{0}$ | 10 |
| 12 | $5.2141973474426608 \times 10^{0}$ | 11 |
| 13 | $6.2057569066466032 \times 10^{0}$ | 12 |
| 14 | $5.8500066418189800 \times 10^{0}$ | 13 |
| 15 | $3.2806512105039980 \times 10^{0}$ | 14 |
| 16 | $1.0145720575982533 \times 10^{0}$ | 15 |

Maximum Error: 26 eps,
observed at order 3.

| | New Approach | |
|---|---|---|
| **I** | **Coefficient Error(eps)** | **Order** |
| 1 | $1.0000000000000000 \times 10^{0}$ | 0 |
| 2 | $2.6250000000000000 \times 10^{0}$ | 1 |
| 3 | $2.6875000000000000 \times 10^{1}$ | 2 |
| 4 | $1.0000000000000000 \times 10^{0}$ | 3 |
| 5 | $2.0195312500000000 \times 10^{0}$ | 4 |
| 6 | $1.0000000000000000 \times 10^{0}$ | 5 |
| 7 | $1.0000000000000000 \times 10^{0}$ | 6 |
| 8 | $1.0000000000000000 \times 10^{0}$ | 7 |
| 9 | $1.0000000000000000 \times 10^{0}$ | 8 |
| 10 | $1.0000000000000000 \times 10^{0}$ | 9 |
| 11 | $1.0000000000000000 \times 10^{0}$ | 10 |
| 12 | $1.0000000000000000 \times 10^{0}$ | 11 |
| 13 | $1.0000000000000000 \times 10^{0}$ | 12 |
| 14 | $1.0000000000000000 \times 10^{0}$ | 13 |
| 15 | $1.0000000000000000 \times 10^{0}$ | 14 |
| 16 | $1.0000000000000000 \times 10^{0}$ | 15 |

Maximum Error: 27 eps,
observed at order 2.

We test the performance of the new approach at $x_0 = 0.58$, which is the transition point between the second branch and the third branch (standard formula for asinc). This is the weakest point for the second branch. As expected, we observe slight numerical inaccuracies; however, these inaccuracies are minimal, and the function remains highly precise. Moreover, the error maintains 'continuity' in the sense defined earlier.

After analyzing the performance in the problematic neighborhood around 0, we observe that the new approach performs exceptionally well, significantly outperforming the previous implementation ($\frac{\operatorname{asin}(x)}{x}$). Once again, the instability caused by the division is eliminated. Furthermore, the new approach also demonstrates robust performance in general tests.

### 3.2.8 Asinhc

For Asinhc, the analysis closely mirrors that for asinc, as the implemented formulas are nearly identical. The only notable difference lies in the derivatives of asinhc, which exhibit a singularity at $x = 1$. This singularity leads to a loss of accuracy near $x = 1$. However the analysis is really similar and, for this reason, we omit it.

## 3.3 Summary

Below, we provide a table summarizing the qualitative improvements achieved in this work.

| **Function** | **Improvements (Qualitative)** |
|---|---|
| Atan | • Improved accuracy in the interval $(0,1)$, particularly for very higher orders.<br>• Enhanced accuracy near the singularities at $x = \pm i$. |
| Acot | • Improved accuracy in the interval $(0,1)$, particularly for very higher orders.<br>• Enhanced accuracy near the singularities at $x = \pm i$. |
| Atanh | • Improved accuracy in the interval $(0,1)$, particularly for very higher orders.<br>• Enhanced accuracy near the singularities at $x = \pm 1$. |
| Acoth | • Improved accuracy in the interval $(0,1)$, particularly for very higher orders.<br>• Enhanced accuracy near the singularities at $x = \pm 1$. |

| **Function** | **Improvements (Qualitative)** |
|---|---|
| Acosh | • Slightly improved accuracy near the singularity at $x = 1$. |
| Faddeeva | • Substantially improved accuracy across the real and complex domains for all orders. |
| Sinc | • Substantially improved accuracy near $x = 0$ for all orders. |
| Sinhc | • Substantially improved accuracy near $x = 0$ for all orders. |
| Asinc | • Substantially improved accuracy near $x = 0$ for all orders. |
| Asinhc | • Substantially improved accuracy near $x = 0$ for all orders.<br>• Enhanced accuracy near the derivative singularity at $x = 1$. |

Table 3.1: Improvements of the work done.

# Chapter 4

# Case Study

In the previous chapter, we demonstrated that the new changes significantly enhance the accuracy and stability of our methods for various functions. It is important to emphasize that GTPSA form the foundation for the entire codebase implementing the physics of MAD-NG. Essentially, the new series implementation are compatible with the framework that handles real numbers, complex numbers, and dual numbers seamlessly, without any changes to the scripts. This highlights the critical role of TPSA (Truncated Power Series Algebra) as the underlying framework. Given its fundamental nature, the precision of this algebra is paramount—imagine a scenario where basic operations like multiplication or addition of real numbers are unreliable.

To illustrate the impact of these improvements, we provide a concrete example of how the enhanced library performs within the *MAD-NG* framework for accelerator beam physics. We start by giving some physical background, in order to understand the scope of the simulation, and then we present the numerical improvements.

## 4.1 Beam-Beam Effect and Twiss Parameters

Here, we demonstrate the improvements made to the library using the specific example of the beam-beam effect. Previously, the implementation of the Faddeeva function led to significant errors in the twiss parameters due to instability in the Faddeeva function starting from order 1.

## 4.2 Beam-Beam Effects: Brief Theoretical Overview

Beam-beam interactions represent a fundamental challenge in particle accelerator beam physics, where two high-energy particle beams interact through electromagnetic forces. These interactions are critical in understanding particle collision dynamics, particularly in advanced research facilities like the Large Hadron Collider (LHC) at CERN.

### 4.2.1 Beam Density Distribution

The mathematical description of beam density provides crucial insights into particle interactions. For bi-Gaussian transverse beam density distributions:

$$\rho(x,y) = \rho_x(x) \cdot \rho_y(y), \tag{4.1}$$

where the one-dimensional density is:

$$\rho_u(u) = \frac{1}{\sigma_u\sqrt{2\pi}} \exp\left(-\frac{u^2}{2\sigma_u^2}\right), \quad u = x,y, \tag{4.2}$$

This distribution represents the probabilistic nature of particle positioning within the beam. The parameter $\sigma_u$ describes the spread of particles in each dimension, capturing the beam's spatial characteristics.

The two-dimensional potential $U(x,y,\sigma_x,\sigma_y)$ is given by [27] [28]:

$$U(x,y,\sigma_x,\sigma_y) = \frac{ne}{4\pi\varepsilon_0} \int_0^\infty \frac{\exp\left(-\frac{x^2}{2\sigma_x^2+q} - \frac{y^2}{2\sigma_y^2+q}\right)}{\sqrt{(2\sigma_x^2+q)\left(2\sigma_y^2+q\right)}}\,\mathrm{d}q, \tag{4.3}$$

where $n$ is the linear particle density, $e$ is the elementary charge, and $\varepsilon_0$ is the permittivity of free space. Specifically the potential is derived solving the Poisson equation (we start by having the charge density).

This complex integral captures the electromagnetic potential created by the particle distribution, accounting for the three-dimensional nature of beam interactions.

The transverse electric field components are derived via:

$$\vec{E} = -\nabla U\left(x, y, \sigma_x, \sigma_y\right). \tag{4.4}$$

### 4.2.2 Elliptical Beam Fields

For elliptical beams ($\sigma_x \neq \sigma_y$), assuming $\sigma_x > \sigma_y$, the transverse fields become more intricate. The asymmetry in beam dimensions leads to complex electromagnetic interactions that require advanced mathematical techniques.

The electric field components become [29] [30]:

$$E_x = \frac{ne}{2\varepsilon_0\sqrt{2\pi(\sigma_x^2-\sigma_y^2)}}\,\mathrm{Im}\left[\mathrm{wf}\left(\frac{x+iy}{\sqrt{2(\sigma_x^2-\sigma_y^2)}}\right) - \mathrm{e}^{-\frac{x^2}{2\sigma_x^2}+\frac{y^2}{2\sigma_y^2}}\,\mathrm{wf}\left(\frac{x\frac{\sigma_y}{\sigma_x}+iy\frac{\sigma_x}{\sigma_y}}{\sqrt{2(\sigma_x^2-\sigma_y^2)}}\right)\right], \tag{4.5}$$

$$E_y = \frac{ne}{2\varepsilon_0\sqrt{2\pi(\sigma_x^2-\sigma_y^2)}}\,\mathrm{Re}\left[\mathrm{wf}\left(\frac{x+iy}{\sqrt{2(\sigma_x^2-\sigma_y^2)}}\right) - \mathrm{e}^{-\frac{x^2}{2\sigma_x^2}+\frac{y^2}{2\sigma_y^2}}\,\mathrm{wf}\left(\frac{x\frac{\sigma_y}{\sigma_x}+iy\frac{\sigma_x}{\sigma_y}}{\sqrt{2(\sigma_x^2-\sigma_y^2)}}\right)\right], \tag{4.6}$$

Here, wf represents the Faddeeva function:

$$\mathrm{wf}(z) = \mathrm{e}^{-z^2}\left[1+\frac{2i}{\sqrt{\pi}}\int_0^z \mathrm{e}^{t^2}\,\mathrm{d}t\right]. \tag{4.7}$$

The corresponding magnetic fields are calculated as:

$$B_y = -\frac{\beta_r E_x}{c}, \quad B_x = \frac{\beta_r E_y}{c},$$

where:

- $\beta_r = \frac{v}{c}$ is the beam velocity as a fraction of the speed of light ,
- $E_x, E_y$ are the components of the electric field,
- $c$ is the speed of light.

These relations arise from the Lorentz transformation of the electric field components due to the

motion of the beam, and the magnetic fields are determined by the cross product $\vec{v} \times \vec{E}$. The signs follow the right-hand rule.

The total Lorentz force on a charged particle emerges from these fields is then:

$$\vec{F} = t(\vec{E} + \vec{v} \times \vec{B}). \tag{4.8}$$

where $t$ is the charge of the particle.

### 4.2.3 Round Beam Fields

For round beams ($\sigma_x = \sigma_y = \sigma$), the potential simplifies to cylindrical coordinates, where $r^2 = x^2 + y^2$:

$$E_r = -\frac{ne}{4\pi\varepsilon_0} \frac{\partial}{\partial r} \int_0^\infty \frac{\exp\left(-\frac{r^2}{2\sigma^2+q}\right)}{2\sigma^2+q} \, \mathrm{d}q, \tag{4.9}$$

$$B_\phi = -\frac{ne\beta c\mu_0}{4\pi} \frac{\partial}{\partial r} \int_0^\infty \frac{\exp\left(-\frac{r^2}{2\sigma^2+q}\right)}{2\sigma^2+q} \, \mathrm{d}q, \tag{4.10}$$

where $\mu_0$ is the magnetic permeability of free space.

The symmetry of round beams allows for a more straightforward mathematical treatment, though the underlying physics remains complex.

### 4.2.4 Recap

From what we analyzed and presented in 4.2.2, we see that a precise computation and representation of the Faddeeva function within our algebra is important to accurately compute the forces that the beams experience when passing through the interaction points. Accurate force computations are essential because small errors in the force calculations can lead to deviations in the beam dynamics. Thus, ensuring precise force calculations is crucial for optimizing the beam's trajectory, minimizing unwanted effects, and maintaining high precision.

## 4.3 Twiss Parameters

The Twiss parameters, introduced by Courant and Snyder [32], provide a convenient way to describe the phase space behavior of particles in linear optics. These parameters are particularly useful when

analyzing beam dynamics near a reference trajectory. They are defined by three functions of position $s$: $\alpha(s)$, $\beta(s)$, and $\gamma(s)$, which parameterize the oscillatory motion of particles in the transverse plane.

The phase space coordinates can be expressed in terms of the Twiss parameters as:

$$\begin{pmatrix} x \\ x' \end{pmatrix} = \begin{pmatrix} \sqrt{\beta(s)} & 0 \\ -\frac{\alpha(s)}{\sqrt{\beta(s)}} & \frac{1}{\sqrt{\beta(s)}} \end{pmatrix} \begin{pmatrix} x_n \\ x_n' \end{pmatrix},$$

where:

- $x$ and $x'$ are the transverse position and angle of the particle at position $s$.
- $x_n$ and $x_n'$ are the normalized coordinates, defined by:

$$x_n = \frac{x}{\sqrt{\beta(s)}} \quad \text{and} \quad x_n' = \sqrt{\beta(s)}x' + \frac{\alpha(s)}{\sqrt{\beta(s)}}x.$$

In the context of the One-Turn Map (OTM), the Twiss parameters allow for a transformation of the beam's dynamics into a normalized phase space. This simplifies the analysis of the beam's motion, which can be represented as a rotation:

$$M = A \cdot R \cdot A^{-1},$$

where:

- $A$ is the transformation matrix containing the Twiss parameters:

$$A = \begin{pmatrix} \sqrt{\beta(s)} & 0 \\ -\frac{\alpha(s)}{\sqrt{\beta(s)}} & \frac{1}{\sqrt{\beta(s)}} \end{pmatrix}.$$

- $R$ is a pure rotation matrix describing the simplified motion:

$$R = \begin{pmatrix} \cos(\Delta\mu) & \sin(\Delta\mu) \\ -\sin(\Delta\mu) & \cos(\Delta\mu) \end{pmatrix},$$

  where $\Delta\mu$ is the phase advance over one turn.

The use of Twiss parameters provides a compact description of the beam's dynamics, reducing the

complexity of phase space analysis and enabling the characterization of the beam's oscillatory behavior in accelerators.

## 4.4 Numerical Simulation

We now provide the results of the effects and improvements in MAD-NG thanks to the introduction of eq. 2.15 and the respective perturbative approach. To check that the results are correct we are going to use the PTC [6] code developed by Etienne Forest. First we explain the code used for the simulation.

```
local sequence, beam, twiss, option in MAD
local quadrupole, drift, wire, beambeam in MAD.element
local clight, qelect in MAD.constant

local qf = quadrupole 'QF' {l=3, k1= 0.04} ! Focusing quadrupole
local qd = quadrupole 'QD' {l=3, k1=-0.04} ! Defocusing quadrupole
local my_wire = wire 'MyWire' {xma=0.01, yma=0, l=0, l_int=100000, l_phy=1, current=90}
local my_bb   = beambeam 'MyBB' {xma=0.01, yma=0, l=0, sigx=1.1e-10, sigy=1e-10, charge=-1, npart=90/(clight*qelect)}

option.nocharge = true

local elm = "bbeam" -- "drift"  "bbeam" "wire"

! Define the FODO cell
local fodo = sequence 'FODO' { l=20, refer="entry", beam = beam {},
  qf      { at= 0 },
  elm == "wire"  and my_wire { at= 5.5 } or
  elm == "bbeam" and my_bb   { at= 5.5, enabled=true } or
  elm == "drift" and drift   { at= 5.5 } or nil,
  qd      { at=10 },
}

local tw = twiss { sequence=fodo, method=2 } --, info=4, debug=4 }
tw:write ("tw_"..elm.."_n.tfs")
```

## Explanation of Lua Code in MAD-NG

The following Lua code is written for *MAD-NG* to define and simulate an accelerator sequence. Here's a detailed breakdown:

```
-- Import necessary modules and constants
local sequence, beam, twiss, option in MAD
local quadrupole, drift, wire, beambeam in MAD.element
local clight, qelect in MAD.constant
```

### Module and Constant Imports

- **Core Simulation Modules**:
  - `sequence`: Defines accelerator beam line structure
  - `beam`: Manages beam properties
  - `twiss`: Performs beam optics analysis
  - `option`: Groups some global parameters
- **Beam Elements**:
  - `quadrupole`: Magnetic focusing/defocusing elements
  - `drift`: Spaces between active elements
  - `wire`: Simulates wire-like interactions
  - `beambeam`: Models particle beam interactions
- **Physical Constants**:
  - `clight`: Speed of light
  - `qelect`: Elementary charge

```
local qf = quadrupole 'QF' {l=3, k1= 0.04} -- Focusing quadrupole
local qd = quadrupole 'QD' {l=3, k1=-0.04} -- Defocusing quadrupole
```

### Quadrupole Magnet Configuration

- **Focusing Quadrupole (QF)**:
  - Length: 3 meters
  - Focusing strength ($k_1$): +0.04
- **Defocusing Quadrupole (QD)**:

- Length: 3 meters
- Defocusing strength ($k_1$): -0.04

```
local my_wire = wire 'MyWire' {xma=0.01, yma=0, l=0,
            l_int=100000, l_phy=1, current=90}
local my_bb   = beambeam 'MyBB' {xma=0.01, yma=0, l=0, sigx=1.1e-10,
            sigy=1e-10, charge=-1, npart=90/(clight*qelect)}
```

### Interaction Elements

- **Wire Element (MyWire)**:
  - Transverse misalignment: (0.01, 0)
  - Physical length: 1 meter
  - Integration length: 100,000 meters
  - Current: 90 amperes
- **Beam-Beam Element (MyBB)**:
  - Transverse misalignment: (0.01, 0)
  - Beam sizes: $\sigma_x = 1.1 \times 10^{-10}$, $\sigma_y = 1.0 \times 10^{-10}$ meters
  - Particle charge: -1
  - Particle density: $\frac{90}{c \cdot q_e}$

```
option.nocharge = true
local elm = "bbeam" -- Options: "drift", "bbeam", "wire"
```

### Simulation Configuration

- Particle charge: Disabled
- Interaction element type: Beam-beam

```
local fodo = sequence 'FODO' { l=20, refer="entry", beam = beam {},
qf      { at= 0 },
```

```
elm == "wire"  and my_wire { at= 5.5 } or
elm == "bbeam" and my_bb   { at= 5.5, enabled=true } or
elm == "drift" and drift   { at= 5.5 } or nil,
qd      { at=10 },
}
```

### FODO Cell Sequence

- Total length: 20 meters
- Reference point: Entry
- Sequence components:
  - Focusing quadrupole at 0 meters
  - Interaction element at 5.5 meters
  - Defocusing quadrupole at 10 meters
- Flexible element insertion based on `elm` variable

```
local tw = twiss { sequence=fodo, method=2 }
tw:write ("tw_"..elm.."_n.tfs")
```

### Twiss Analysis

- Performs Twiss parameter calculation
- Integration method at order 2
- Writes results to file `tw_<elm>_n.tfs`

The results of the following code for the twiss parameters were totally different with respects to the ones presented by PTC. Indeed for PTC we have

| Pos | $\alpha_x$ | $\beta_x$ | $\gamma_x$ | $\mu_x$ | $\alpha_y$ | $\beta_y$ | $\gamma_y$ | $\mu_y$ |
|---|---|---|---|---|---|---|---|---|
| Start | -5.742653497 | 119.6560867 | 0.2839644027 | 0 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0 |
| Focusing | 8.409855138 | 110.5324326 | 0.6489105662 | 0.003907464564 | -0.9497266988 | 9.060169782 | 0.2099277219 | 0.06226512427 |
| Drift | 6.787578723 | 72.53884798 | 0.6489105662 | 0.008351590813 | -1.474546064 | 15.12085154 | 0.2099277219 | 0.09652317876 |
| BeamBeam | 2.873190295 | 72.53884798 | 0.127589874 | 0.008351590813 | -0.6585847921 | 15.12085154 | 0.09481833247 | 0.09652317876 |
| Drift | 2.299035862 | 49.26383028 | 0.127589874 | 0.02034366347 | -1.085267288 | 22.9681859 | 0.09481833247 | 0.1353377536 |
| Defocusing | -3.754902679 | 53.17319349 | 0.2839644027 | 0.03024886376 | 1.526533702 | 21.45875292 | 0.1551956516 | 0.1556321268 |
| Drift | -5.742653497 | 119.6560867 | 0.2839644027 | 0.04423389456 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0.2473395301 |
| End | -5.742653497 | 119.6560867 | 0.2839644027 | 0.04423389456 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0.2473395301 |

Table 4.1: Twiss parameters obtained through PTC with an equivalent script as the one presented.

| Pos | $\alpha_x$ | $\beta_x$ | $\gamma_x$ | $\mu_x$ | $\alpha_y$ | $\beta_y$ | $\gamma_y$ | $\mu_y$ |
|---|---|---|---|---|---|---|---|---|
| Start | 0.1339137681 | 10.22730759 | 0.09953087736 | 0 | 0.7139898941 | 7.123926248 | 0.211931106 | 0 |
| Focusing | 0.769833064 | 7.135875221 | 0.223188172 | 0.05360455222 | -0.5822938966 | 6.770289856 | 0.1977856503 | 0.07578207114 |
| Drift | 0.211862634 | 4.681635977 | 0.223188172 | 0.1247943074 | -1.076758022 | 10.91791965 | 0.1977856503 | 0.1227396694 |
| BeamBeam | 1.203016577e+16 | 4.681635977 | 3.091331518e+31 | 0.1247943074 | -2.865523197e+16 | 10.91791965 | 7.209212614e+31 | 0.1227396694 |
| Drift | -1.391099183e+32 | 6.259946324e+32 | 3.091331518e+31 | 0.6247943074 | -3.244145676e+32 | 1.459865554e+33 | 7.209212614e+31 | 0.1227396694 |
| Defocusing | -4.630277553e+32 | 2.242883517e+33 | 9.558887056e+31 | 0.6247943074 | -1.494706569e+32 | 3.08087718e+33 | 7.251661129e+30 | 0.1227396694 |
| Drift | -1.132149849e+33 | 1.340912675e+34 | 9.558887056e+31 | 0.6247943074 | -2.002322848e+32 | 5.528797772e+33 | 7.251661129e+30 | 0.1227396694 |
| End | -1.132149849e+33 | 1.340912675e+34 | 9.558887056e+31 | 0.6247943074 | -2.002322848e+32 | 5.528797772e+33 | 7.251661129e+30 | 0.1227396694 |

Table 4.2: Twiss parameters obtained by MAD-NG with the previous implementation of the Faddeva function. As one can observe, the values are not only different than the ones obtained through PTC but they are also diverging.

| Pos | $\alpha_x$ | $\beta_x$ | $\gamma_x$ | $\mu_x$ | $\alpha_y$ | $\beta_y$ | $\gamma_y$ | $\mu_y$ |
|---|---|---|---|---|---|---|---|---|
| Start | -5.742653497 | 119.6560867 | 0.2839644027 | 0 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0 |
| Focusing | 8.409855138 | 110.5324326 | 0.6489105662 | 0.003907464564 | -0.9497266988 | 9.060169782 | 0.2099277219 | 0.06226512427 |
| Drift | 6.787578723 | 72.53884798 | 0.6489105662 | 0.008351590813 | -1.474546084 | 15.12085154 | 0.2099277219 | 0.09652317876 |
| BeamBeam | 2.873190295 | 72.53884798 | 0.127589874 | 0.008351590813 | -0.6585847921 | 15.12085154 | 0.09481833247 | 0.09652317876 |
| Drift | 2.299035862 | 49.26383028 | 0.127589874 | 0.02034366347 | -1.085267288 | 22.9681859 | 0.09481833247 | 0.1353377536 |
| Defocusing | -3.754902679 | 53.17319349 | 0.2839644027 | 0.03024886376 | 1.526533702 | 21.45875292 | 0.1551956516 | 0.1556321268 |
| Drift | -5.742653497 | 119.6560867 | 0.2839644027 | 0.04423389456 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0.2473395301 |
| End | -5.742653497 | 119.6560867 | 0.2839644027 | 0.04423389456 | 0.4401641404 | 7.691868025 | 0.1551956516 | 0.2473395301 |

Table 4.3: Twiss parameters obtained by MAD-NG with the new implementation of the Faddeva function. As we can see, we get the same exact values obtained through PTC.

Looking at the different values of the Twiss parameters, we immediately notice that the new stable implementation of the Faddeva function is in complete agreement with the values obtained by PTC: in tab. 4.3 we obtain the exact same values obtained in tab. 4.1 through the equivalent simulation using PTC. Instead, in tab. 4.2 we see the values obtained through the previous implementation of the Faddeeva function are not correct.

## 4.5 Summary

We have observed that the new computational approach for the Faddeeva function significantly improves simulation accuracy in the context of the recently introduced (in *MAD-NG*) beam-beam effect and based on its analytical relationship with the error function. However, it is essential to highlight that the advantages of these updated functions extend beyond specific cases.

When working with a simulation tool, having thoroughly tested mathematical and numerical methods ensures that we are aware of their limitations and domain of validity. This allows us to simulate any physical phenomenon without questioning the reliability of the numerical results. In the worst-case scenario, we might need to question the validity of the underlying model, but not the numerical framework.

# Chapter 5

# Conclusion

In this work, we presented a technique for automatic differentiation based on dual number algebra and successfully discovered and validated new analytical series to improve the accuracy of numerical implementations. Through our research, we described the power of this algebraic approach in studying physical systems that exhibit complex and non-linear behavior, which can be naturally analyzed using this framework.

The first phase of our work focused on identifying instabilities in functions and establishing a reliable data generation procedure for their validation.

The second phase presented our most significant challenge, as we worked on developing stable representations of certain problematic functions within the dual number algebra. To address this, we successfully derived new analytical and perturbative series, providing more accurate and computationally stable function representations. The significance of these series extends beyond our specific implementation, as they offer valuable tools for general analytical calculations in related fields.

Finally, we concentrated on validating and numerically testing the new functions. The results definitively demonstrated our success in establishing stable representations of previously problematic functions within the dual number framework.

In conclusion, we successfully completed the GTPSA package in *MAD-NG*, establishing it as one of the most powerful and accurate open-source libraries for differential algebraic calculations. The package's precision and computational efficiency make it an invaluable tool for advancing perturbative analyses across multiple domains.

# Bibliography


[1] M. Berz, *High-order computation and normal form analysis of repetitive systems*, AIP Conf. Proc., vol. 249, pp. 456–489, Mar. 1992, DOI: https://doi.org/10.1063/1.41975.

[2] berz1989 M. Berz, *Differential algebraic description of beam dynamics to very high orders*, Particle Accelerators, vol. 24, pp. 109–124, 1989, Gordon and Breach Science Publishers, Inc.

[3] M.Berz, *Modern Map Methods in Particle Beam Physics*, Academic Press, 1999, ISBN: 978-0-12-014750-2.

[4] A.J.Dragt, *Lie Algebraic Methods for Charged Particle Optics*, AIP Conference Proceedings, vol. 177, no. 1, pp. 261–264, 1988, ISBN: 0-88318-377-3.

[5] L. Deniau, *MAD-NG - a Standalone Multiplatform Tool for Linear and Non-linear Optics Design and Optimisation*, presented at the 14th International Computational Accelerator Physics Conference (ICAP 2024), 2-5 October 2024, Seeheim-Jugenheim, Germany, CERN-BE/ABP. Available: https://arxiv.org/abs/2412.16006.

[6] E. Forest, F. Schmidt, and E. McIntosh, *Introduction to the Polymorphic Tracking Code: Fibre Bundles, Polymorphic Taylor Types and "Exact Tracking"*, CERN–SL–2002–044 (AP), KEK-Report 2002-3, National High Energy Research Organization (KEK), Japan, CERN, Geneva, CH, 2002. Available: https://cds.cern.ch/record/573082/files/sl-2002-044.pdf.

[7] M. Berz and K. Makino, *COSY INFINITY 10.2*, BMT Dynamics, April 2023. Available: https://bmtdynamics.org.

[8] D.T.Abell, E.McIntosh, and F.Schmidt, *Fast Symplectic Map Tracking for the CERN Large Hadron Collider*, Physical Review Special Topics - Accelerators and Beams, vol. 6, no. 6, June 2003. Available at: https://doaj.org/article/08116996a6e84824a292453d4bfc52c6.

[9] C.Tomoiagă and L.Deniau, *Generalised Truncated Power Series Algebra for Fast Particle Accelerator Transport Maps*, in Proceedings of the 6th International Particle Accelerator Conference (IPAC'15), Richmond, VA, USA, JACoW Publishing, 2015, ISBN: 978-3-95450-168-7, DOI: 10.18429/JACoW-IPAC2015-MOPJE039.

[10] V.Ziemann, *Beyond Bassetti and Erskine: Beam-Beam Deflections for Non-Gaussian Beams*, Stanford Linear Accelerator Center, Stanford University, Stanford, CA 94309, 1994.

[11] W.Herr, CERN Accelerator School, Trondheim, Norway, 2013. Available: `https://cas.web.cern.ch/sites/default/files/lectures/trondheim-2013/herr2.pdf`.

[12] W.Herr and E. Forest, *Non-linear Dynamics in Accelerators*, in *Particle Physics Reference Library*, pp. 51-104, Springer, 2020, DOI: 10.1007/978-3-030-34245-6_3.

[13] E.Courant and H. Snyder, *Theory of the Alternating Gradient Synchrotron*, Ann. Phys., vol. 3, pp. 1–48, 1958.

[14] A.Wolski, *Beam Dynamics in High Energy Particle Accelerators*, Imperial College Press, 2014.

[15] H.Wiedemann, *Particle accelerator beam physics —Basic Principles and Linear Beam Dynamics*, Springer-Verlag, 1993.

[16] W.Herr, *Mathematical and Numerical Methods for Non-linear Beam Dynamics*, CERN Yellow Report CERN-2014-009, pp. 157-198, 2014. Available: `https://doi.org/10.5170/CERN-2014-009.157`, arXiv:1601.05411 [physics.acc-ph], DOI: 10.48550/arXiv.1601.05411.

[17] W.Herr, *Nonlinear Dynamics - Methods and Tools — and a Contemporary (1985+) Framework to Treat Linear and Nonlinear Beam Dynamics, Part 1,3*, London, 11 Sept. 2017. Available: `https://cas.web.cern.ch/sites/default/files/lectures/egham-2017/nld1.pdf` and `https://cas.web.cern.ch/sites/default/files/lectures/egham-2017/nld3.pdf`.

[18] J. Fike, S. Jongsma, and J. Alonso, *Optimization with Gradient and Hessian Information Calculated Using Hyper-Dual Numbers*, 29th AIAA Applied Aerodynamics Conference, 2011. Available: `https://doi.org/10.2514/6.2011-8861`.

[19] V. Brodsky and M. Shoham, *Dual Numbers Representation of Rigid Body Dynamics*, Mechanism and Machine Theory, vol. 34, no. 5, pp. 693-703, 1999. Available: `https://doi.org/10.1016/S0094-114X(98)00050-1`.

[20] J. Fike and J. Alonso, *The Development of Hyper-Dual Numbers for Exact Second-Derivative Calculations*, 49th AIAA Aerospace Sciences Meeting, 2011. Available: https://doi.org/10.2514/6.2011-8860.

[21] Y. L. Gu and J. Luh, *Dual-Number Transformation and Its Applications to Robotics*, IEEE Journal on Robotics and Automation, vol. 3, no. 6, pp. 615-623, Dec. 1987. Available: https://doi.org/10.1109/JRA.1987.1087137.

[22] A. Cohen and M. Shoham, *Application of Hyper-Dual Numbers to Multibody Kinematics*, Journal of Mechanisms and Robotics, vol. 8, no. 3, pp. 031012, 2016. Available: https://doi.org/10.1115/1.4032968.

[23] T. Wei, W. Ding, and Y. Wei, *Singular Value Decomposition of Dual Matrices and Its Application to Traveling Wave Identification in the Brain*, SIAM Journal on Matrix Analysis and Applications, vol. 45, no. 1, pp. 12-34, 2024. Available: https://doi.org/10.1137/21M1451035.

[24] A. Pal, F. Holtorf, A. Larsson, T. Loman, and F. Schäfer, *Nonlinearsolve.jl: High-Performance and Robust Solvers for Systems of Nonlinear Equations in Julia*, arXiv preprint, 2024. Available: https://arxiv.org/abs/2401.04567.

[25] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*, National Bureau of Standards, 1964. Available: https://doi.org/10.6028/NBS.AMS.55.

[26] M. Berz and K. Makino, *Verified Integration of ODEs and Flows Using Differential Algebraic Methods on High-Order Taylor Models*, Reliable Computing, vol. 4, no. 4, pp. 361–369, 1998.

[27] O. D. Kellogg, *Foundations of Potential Theory*, Dover, New York, 1953.

[28] B. Houssais, *Thesis*, University of Rennes, 1967.

[29] J. E. Augustin, *Turn by Turn Study of $e^+e^-$ Space Charge Effects*, SLAC, 1973.

[30] M. Bassetti and G. A. Erskine, *CERN-ISR-TH/80-06*, 1980.

[31] E. Keil, *Beam-beam dynamics*, CERN-SL-94-78-AP, CERN, 1995. In: *CAS - CERN Accelerator School: 5th Advanced accelerator beam physics Course*, pp. 539-556, 20 Sep 1994. DOI: 10.5170/CERN-1995-006.539.

[32] E. D. Courant and H. S. Snyder, *Theory of the Alternating-Gradient Synchrotron*, Annals of Physics, vol. 3, pp. 1–48, 1958. DOI: 10.1016/0003-4916(58)90012-5.

# Appendix A

# Data generation code

## A.1 Mathematica code

```
(* Define the sinhc function *)
sinhc[x_] := Piecewise[{{Sinh[x]/x, x != 0}}, 1]

(* Define the Eform function *)
Eform[x_?NumericQ, ndig_Integer : 20] :=
 Module[{u, s, p, base, exp, sign, result},
  u = If[x == 0, 0, x];
  {s, p} = MantissaExponent[u];
  If[s != 0, {s = s*10; p = p - 1}];
  base = ToString[PaddedForm[s, {ndig + 2, ndig}]];
  exp = If[p >= 0, ToString[p], ToString[-p]];
  If[StringLength[exp] < 2, exp = StringJoin["0", exp], exp = exp];
  sign = If[p >= 0, "e+", "e-"];
  result = StringJoin[base, sign, exp];
  result]

(* Compute the series coefficients of sinhc *)
data1 = N[
  Table[SeriesCoefficient[sinhc[x], {x, Pi/3, n}], {n, 0, 15}], 25]

(* Format the coefficients using Eform *)
formattedData = Eform /@ data1

(* Export the formatted data to a file *)
Export["data1.dat", formattedData]

(* Display the content of the file *)
FilePrint["data1.dat"]
```

Listing A.1: Code used to generate data with Mathematica.

This Mathematica code is a general-purpose script designed to compute and format the coefficients of the series expansion of a given function around a specified point. It allows the user to change the function being expanded and the point of evaluation as needed. The steps are as follows:

1. **Define the Function to Expand**: The script defines the function of interest. In this example, the hyperbolic sine cardinal function, $\text{sinhc}(x) = \frac{\sinh(x)}{x}$, is implemented using a piecewise definition to handle the special case where $x = 0$.

2. **Customize Scientific Notation (`Eform`)**: A custom formatting function, `Eform`, is defined to express numeric values in scientific notation with a specified number of significant digits. This ensures the precision and readability of the computed coefficients.

3. **Compute Series Coefficients**: Using the `SeriesCoefficient` function, the code calculates the coefficients of the Taylor series expansion of the defined function around a chosen point (in this example, $x = \frac{\pi}{3}$) up to the 15th term.

4. **Format and Export the Coefficients**: The coefficients are formatted using the `Eform` function and exported to a file named `data1.dat` for storage or further use.

5. **Display Exported Data**: The content of the exported file is printed for verification.

By modifying the function definition and the expansion point in the code, this script can be adapted for a wide range of functions and series expansions, making it a flexible tool for numerical and analytical computations.

## A.2 Sympy code

```
import sympy as sp
from sympy import I

def compute_and_evaluate_derivatives(x0_points, n, fun_expr, precision=25):
    """
    Computes Taylor series coefficients up to order n for a function expanded around each x0 in x0_points.

    Parameters:
    - x0_points: List of points at which to expand the Taylor series.
    - n: Order up to which to compute the Taylor series.
    - fun_expr: The function expression to expand (e.g., sp.asin(x)/x).
    - precision: Number of decimal places for the output.

    Returns:
    - List of lists of derivatives evaluated at each expansion point.
    """
    x = sp.symbols('x')
    all_derivatives = []
```

```
    for x0 in x0_points:
        derivatives_at_x0 = []
        for i in range(n + 1):
            # Compute the i-th derivative of the function
            derivative = sp.diff(fun_expr, x, i)

            # Divide by factorial(i)
            derivative /= sp.factorial(i)

            # Evaluate the derivative at x0
            derivative_at_x0 = derivative.subs(x, x0).evalf(precision + 5)  # Extra precision
            derivatives_at_x0.append(derivative_at_x0)

        all_derivatives.append(derivatives_at_x0)

    return all_derivatives

def format_x0_as_pi_notation(x0):
    """
    Formats x0 in terms of multiples of pi if applicable, otherwise formats as a number.
    """
    if sp.simplify(x0 / sp.pi).is_rational:
        coeff = sp.simplify(x0 / sp.pi)
        return f"pi*{coeff}".replace('*1/', '/') if coeff != 1 else "pi"
    else:
        # Handle complex numbers
        real_part = sp.re(x0)
        imag_part = sp.im(x0)

        real_str = (f"pi*{sp.simplify(real_part / sp.pi)}" if sp.simplify(real_part / sp.pi).is_rational
                    else str(real_part.evalf())).replace('*1/', '/')
        imag_str = (f"pi*{sp.simplify(imag_part / sp.pi)}" if sp.simplify(imag_part / sp.pi).is_rational
                    else str(imag_part.evalf())).replace('*1/', '/')

        if real_part == 0:
            return f"{imag_str}*1i"
        elif imag_part == 0:
            return real_str
        else:
            return f"{real_str} + {imag_str}*1i" if imag_part > 0 else f"{real_str} - {imag_str[1:]}*1i"

def write_coefficients_to_file(all_derivatives, x0_points, filename, function_name):
    """
    Writes Taylor series coefficients to a file for multiple expansion points.

    Parameters:
    - all_derivatives: List of derivatives evaluated at each x0.
    - x0_points: List of expansion points.
    - filename: Output file path.
    - function_name: Name of the function for the file header.
    """
    with open(filename, 'w') as file:
        # Write the header with the function name
        file.write(f"M.fun.{function_name} = {{\n")

        for i, derivatives in enumerate(all_derivatives):
            x0 = x0_points[i]

            # Format x0 in pi notation if possible
            x0_str = format_x0_as_pi_notation(x0)
            file.write(f"{{ x0 = {x0_str},\n")

            for j, derivative in enumerate(derivatives):
                if j % 2 == 0 and j != 0:
                    file.write("\n")

                # Check if the derivative is real or complex
                if derivative.is_real:
                    # Write the real derivative in scientific notation
                    derivative_str = f"{float(derivative.evalf()):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')
                else:
                    # Format the complex derivative
```

```
                    real_part = sp.re(derivative).evalf()
                    imag_part = sp.im(derivative).evalf()

                    real_str = f"{float(real_part):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')
                    imag_str = f"{float(imag_part):.25e}".replace('e+', 'e+0').replace('e-', 'e-0')

                    if real_part != 0:
                        derivative_str = f"{real_str} + {imag_str}i" if imag_part >= 0 else f"{real_str} - {imag_str[1:]}i"
                    else:
                        derivative_str = f"{imag_str}i" if imag_part >= 0 else f"-{imag_str[1:]}i"

                file.write(f"  {derivative_str},")

            file.write("\n},\n\n")

        file.write("}\n")

# Example usage:
if __name__ == "__main__":
    # Set of x0 points
    x0_points = [
         10,
    ]

    # Order of derivatives
    n = 15

    # Number of decimal places
    precision = 25

    x = sp.symbols('x')

    # Define the Faddeeva function using its integral representation
    t = sp.symbols('t')
    integral_part = sp.integrate(sp.exp(t**2), (t, 0, x))
    fun_expr = sp.exp(-x**2) * (1 + (2 * sp.I / sp.sqrt(sp.pi)) * integral_part)

    # Compute and evaluate the derivatives at the specified points
    all_derivatives = compute_and_evaluate_derivatives(x0_points, n, fun_expr, precision)

    # Output filename
    output_filename = 'C:\\Users\\anton\\Downloads\\function_derivatives_at_points.txt'

    # Specify the name of the function
    function_name = "wf"

    # Write all derivatives to the file
    write_coefficients_to_file(all_derivatives, x0_points, output_filename, function_name)
    print(f"All derivatives written to {output_filename}")

    print("All files generated successfully.")
```

---

The provided Python code performs symbolic computation to generate the Taylor series coefficients of a given function at specified expansion points. It is composed of several functions:

## 1. `compute_and_evaluate_derivatives`

This function computes the coefficients of the Taylor series up to a specified order $n$ for a given symbolic function.

- **Inputs:**
  - `x0_points`: A list of points at which the function is expanded.
  - `n`: The maximum order of derivatives to compute.
  - `fun_expr`: The symbolic expression of the function.
  - `precision`: The number of decimal places for the output (default is 25).
- **Process:**
  - For each expansion point $x_0$, compute up to the $i$-th coefficient of the taylor expansion of the function.
  - Evaluate the coefficients at $x_0$ and store it with extended precision.
- **Output:** A list of Taylor series coefficients for each expansion point.

### 2. `format_x0_as_pi_notation`

This function formats the expansion points $x_0$ in terms of multiples of $\pi$, if possible.

- **For Real Points:** If $x_0$ is a rational multiple of $\pi$, it is expressed in terms of $\pi$. Otherwise, it is formatted as a numerical value.
- **For Complex Points:** The real and imaginary parts are separately formatted, with the imaginary part appended as $+i$ or $-i$.

### 3. `write_coefficients_to_file`

This function writes the computed Taylor series coefficients into a file.

- **Process:**
  - Each expansion point is labeled using its formatted representation (e.g., in terms of $\pi$).
  - Coefficients are written in scientific notation, with special handling for complex values (real and imaginary parts are separated).
- **Output:** A structured file where each expansion point and its coefficients are clearly listed. The format to save the data is chosen to respect the test suite for we are using (in Lua).

### Code Workflow

1. Define the symbolic function (`fun_expr`) using SymPy.
2. Call `compute_and_evaluate_derivatives` to compute Taylor coefficients for the function at specified points.
3. Format the expansion points using `format_x0_as_pi_notation`.
4. Save the results to a file using `write_coefficients_to_file`.

This is the main code used to generate the data with an accuracy of 25 digits as a default value. After generating the data, they are compared with the ones generated by Mathematica and then we test our numerical implementations.

# Appendix B

# Functions'code

## B.1 Atan

In this part of the appendix we present the code used in c to implement the formulas derived in chapter 2. The code will be briefly commented focusing on the main features.

```
1  void
2  FUN(atan) (const T *a, T *c)                    // checked for real and complex
3  {
4    assert(a && c); DBGFUN(->);
5    ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6    NUM a0 = a->coef[0], f0 = atan(a0);
7
8    ord_t to = c->mo;
9    if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
10
11 #if OLD_SERIES
12   if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
13     // atan(x) = -i/2 ln((i-x) / (i+x)) ; pole = +/- 1i
14 #ifdef MAD_CTPSA_IMPL
15     ctpsa_t *tn = GET_TMPX(c), *td = GET_TMPX(c);
16     mad_ctpsa_axpb(-1, a, I, tn);
17     mad_ctpsa_axpb( 1, a, I, td);
18     mad_ctpsa_logxdy(tn, td, c);
19     mad_ctpsa_scl(c, -I/2, c);
20 #else
21     ctpsa_t *tn = GET_TMPC(c), *td = GET_TMPC(c);
22     mad_ctpsa_cplx(a, NULL, td);
23     mad_ctpsa_axpb(-1, td, I, tn);
24     mad_ctpsa_seti(td, 0, 1, I);
25     mad_ctpsa_logxdy(tn, td, tn);
26     mad_ctpsa_scl(tn, -I/2, tn);
27     mad_ctpsa_real(tn, c);
28 #endif
29     REL_TMPC(td), REL_TMPC(tn); DBGFUN(<-); return;
30   }
31 #endif // OLD_SERIES
32
33   NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/(1+a2), f2 = f1*f1, f4 = f2*f2;
34   // fill orders <= MANUAL_EXPANSION_ORD
```

```
35      switch(MIN(to, MANUAL_EXPANSION_ORD)) {
36      case 6: ord_coef[6] = -a0*(1 + a2*(-10./3 + a2)) *f4*f2; /* FALLTHRU */
37      case 5: ord_coef[5] = (1./5 + a2*(-2 + a2)) *f4*f1;      /* FALLTHRU */
38      case 4: ord_coef[4] = -a0*(-1 + a2) *f4;                 /* FALLTHRU */
39      case 3: ord_coef[3] = (-1./3 + a2) *f2*f1;               /* FALLTHRU */
40      case 2: ord_coef[2] = -a0 *f2;                           /* FALLTHRU */
41      case 1: ord_coef[1] = f1;                                /* FALLTHRU */
42      case 0: ord_coef[0] = f0;                                break;
43      assert(!"unexpected missing coefficients");
44      }
45      // fill orders > MANUAL_EXPANSION_ORD
46      for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
47        ord_t n = (o-3)/2+1;
48        int s = 2*(o & 1)-1; // o: even = -1, odd = 1
49        NUM v = 0;
50        for (ord_t i = 0; i <= n; ++i, s = -s) {
51          num_t c = s * mad_num_powi(2,o-2*i-1) * mad_num_binom(o-i-1,i);
52          v += c * NUMF(div)(NUMF(powi)(a0,o-2*i-1), NUMF(powi)(1+a2,o-i));
53        }
54        ord_coef[o] = v/o;
55      }
56
57      fun_taylor(a,c,to,ord_coef);
58      DBGFUN(<-);
59    }
60    }
```

- **Initialization and Basic Setup:** The function first computes $f_0 = \text{atan}(a_0)$, where $a_0$ is the real part of the dual number given in input.
- **Manual Expansion of the Taylor Series:** If the Taylor series order $to$ is less than or equal to a predefined limit MANUAL_EXPANSION_ORD, which is set to 6, the function computes the Taylor coefficients manually. These coefficients are stored in the array ord_coef.
- **Old Approach (OLD_SERIES):** If the series order exceeds MANUAL_EXPANSION_ORD, the computation follows the identity:

$$\text{atan}(x) = -\frac{i}{2}\ln\left(\frac{i-x}{i+x}\right)$$

- **New Approach:** For Taylor series orders greater than MANUAL_EXPANSION_ORD, the formula presented in Chapter 2 (see Equation eq. 2.3) is applied. The coefficients are calculated and stored in the array ord_coef. This is specifically done in the last 2 'for' loops.
- **Conclusion:** After calculating the Taylor series coefficients stored in ord_coef, the function evaluates the series as a truncated power series using the function fun_taylor. The output is then stored in $c$.

## B.2 Acot

In this part of the appendix, we present the code used in C to implement the formulas derived for the inverse cotangent (acot) function. The code is briefly commented to focus on the key features and improvements made.

```
void
FUN(acot) (const T *a, T *c)                    // checked for real and complex
{
  assert(a && c); DBGFUN(->);
  ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
  NUM a0 = a->coef[0];
  ensure(a0 != 0, "invalid domain acot("FMT")", VAL(a0));

  NUM f0 = atan(NUMF(inv)(a0));
  ord_t to = c->mo;

#if OLD_SERIES
  if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }

  if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
    // acot(x) = i/2 ln((x-i) / (x+i))
#ifdef MAD_CTPSA_IMPL
    ctpsa_t *tn = GET_TMPX(c), *td = GET_TMPX(c);
    mad_ctpsa_copy( a, tn);
    mad_ctpsa_copy(tn, td);
    mad_ctpsa_seti(tn, 0, 1, -I);
    mad_ctpsa_seti(td, 0, 1,  I);
    mad_ctpsa_logxdy(tn, td, c);
    mad_ctpsa_scl(c, I/2, c);
#else
    ctpsa_t *tn = GET_TMPC(c), *td = GET_TMPC(c);
    mad_ctpsa_cplx(a, NULL, tn);
    mad_ctpsa_copy(tn, td);
    mad_ctpsa_seti(tn, 0, 1, -I);
    mad_ctpsa_seti(td, 0, 1,  I);
    mad_ctpsa_logxdy(tn, td, tn);
    mad_ctpsa_scl(tn, I/2, tn);
    mad_ctpsa_real(tn, c);
#endif
    REL_TMPC(td), REL_TMPC(tn); DBGFUN(<-); return;
  }
#endif // OLD_SERIES

  NUM ord_coef[to+1], a2 = a0*a0, f1 = -1/(1+a2), f2 = f1*f1, f4 = f2*f2;
  // fill orders <= MANUAL_EXPANSION_ORD
  switch(MIN(to, MANUAL_EXPANSION_ORD)) {
  case 6: ord_coef[6] = a0*(1 + a2*(-10./3 + a2)) *f4*f2; /* FALLTHRU */
  case 5: ord_coef[5] = (1./5 + a2*(-2 + a2)) *f4*f1;     /* FALLTHRU */
  case 4: ord_coef[4] = a0*(-1 + a2) *f4;                 /* FALLTHRU */
  case 3: ord_coef[3] = (-1./3 + a2) *f2*f1;              /* FALLTHRU */
  case 2: ord_coef[2] = a0 *f2;                           /* FALLTHRU */
  case 1: ord_coef[1] = f1;                               /* FALLTHRU */
  case 0: ord_coef[0] = f0;                               break;
  assert(!"unexpected missing coefficients");
  }
  // fill orders > MANUAL_EXPANSION_ORD
  for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
    ord_t n = (o-3)/2+1;
    int s = 2*(o & 1)-1; // o: even = -1, odd = 1
    NUM v = 0;
    for (ord_t i = 0; i <= n; ++i, s = -s) {
      num_t c = s * mad_num_powi(2,o-2*i-1) * mad_num_binom(o-i-1,i);
      v += c * NUMF(div)(NUMF(powi)(a0,o-2*i-1), NUMF(powi)(1+a2,o-i));
    }
```

```
60        ord_coef[o] = -v/o;
61      }
62
63      fun_taylor(a,c,to,ord_coef);
64      DBGFUN(<-);
65    }
```

- **Initialization and Basic Setup:** The function first computes $f_0 = \text{atan}(1/a_0)$, where $a_0$ is the real part of the dual number given in input. This value is the initial estimate for the inverse cotangent function.
- **Manual Expansion of the Taylor Series:** If the Taylor series order $to$ is less than or equal to a predefined limit MANUAL_EXPANSION_ORD, set to 6, the function computes the Taylor coefficients manually. These coefficients are stored in the array ord_coef.
- **Old Approach (OLD_SERIES):** If the series order exceeds MANUAL_EXPANSION_ORD, the computation follows the identity:

  $$\text{acot}(x) = \frac{i}{2} \ln\left(\frac{x-i}{x+i}\right)$$

  This allows for a more direct approach using complex logarithms for the inverse cotangent.
- **New Approach:** For Taylor series orders greater than MANUAL_EXPANSION_ORD, the formula derived in Chapter 2 is applied, calculating the necessary coefficients and storing them in the array ord_coef.
- **Conclusion:** After computing the Taylor series coefficients, the function evaluates the series using a truncated power series approach, applying the function fun_taylor. The final result is stored in $c$, the output.

## B.3 Atanh

In this part of the appendix, we present the code used in C to implement the formulas derived for the inverse hyperbolic tangent (`atanh`) function. The code is briefly commented to focus on the key features and improvements made.

```
1  void
2  FUN(atanh) (const T *a, T *c)                    // checked for real and complex
3  {
4    assert(a && c); DBGFUN(->);
```

```
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6     NUM a0 = a->coef[0];
7     ensure(fabs(a0) SELECT(< 1, != 1), "invalid domain atanh("FMT")", VAL(a0));
8     NUM f0 = atanh(a0);
9
10    ord_t to = c->mo;
11    if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
12
13  #if OLD_SERIES
14    if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
15      // atanh(x) = 1/2 ln((1+x) / (1-x))
16      T *tn = GET_TMPX(c), *td = GET_TMPX(c);
17      FUN(copy)(a, tn);
18      FUN(seti)(tn, 0, 1, 1);
19      FUN(axpb)(-1, a, 1, td);
20      FUN(logxdy)(tn, td, c);
21      FUN(scl)(c, 0.5, c);
22      REL_TMPX(td), REL_TMPX(tn); DBGFUN(<-); return;
23    }
24  #endif // OLD_SERIES
25
26    NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/(1-a2), f2 = f1*f1, f4 = f2*f2;
27    // fill orders <= MANUAL_EXPANSION_ORD
28    switch(MIN(to, MANUAL_EXPANSION_ORD)) {
29    case 6: ord_coef[6] = a0*(1 + a2*(10./3 + a2)) *f4*f2; /* FALLTHRU */
30    case 5: ord_coef[5] = (1./5 + a2*(2 + a2)) *f4*f1;     /* FALLTHRU */
31    case 4: ord_coef[4] = a0*(1 + a2) *f4;                 /* FALLTHRU */
32    case 3: ord_coef[3] = (1./3 + a2) *f2*f1;              /* FALLTHRU */
33    case 2: ord_coef[2] = a0 *f2;                          /* FALLTHRU */
34    case 1: ord_coef[1] = f1;                              /* FALLTHRU */
35    case 0: ord_coef[0] = f0;                              break;
36    assert(!"unexpected missing coefficients");
37    }
38    // fill orders > MANUAL_EXPANSION_ORD
39    for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
40      ord_t n = (o-3)/2+1;
41      int s = 2*(o & 1)-1; // o: even = -1, odd = 1
42      NUM v = 0;
43      for (ord_t i = 0; i <= n; ++i, s = -s) {
44        num_t c = s * mad_num_powi(2,o-2*i-1) * mad_num_binom(o-i-1,i);
45        v += c * NUMF(div)(NUMF(powi)(a0,o-2*i-1), NUMF(powi)(a2-1,o-i));
46      }
47      ord_coef[o] = -v/o;
48    }
49
50    fun_taylor(a,c,to,ord_coef);
51    DBGFUN(<-);
52  }
```

- **Initialization and Basic Setup:** The function first computes $f_0 = \text{atanh}(a_0)$, where $a_0$ is the real part of the dual number given in input. This value is the initial estimate for the inverse hyperbolic tangent function.
- **Manual Expansion of the Taylor Series:** If the Taylor series order *to* is less than or equal to a predefined limit MANUAL_EXPANSION_ORD, set to 6, the function computes the Taylor coefficients manually. These coefficients are stored in the array ord_coef.
- **Old Approach (OLD_SERIES):** If the series order exceeds MANUAL_EXPANSION_ORD,

the computation follows the identity:

$$\operatorname{atanh}(x) = \frac{1}{2}\ln\left(\frac{1+x}{1-x}\right)$$

This allows for a more direct approach using logarithms for the inverse hyperbolic tangent.

- **New Approach:** For Taylor series orders greater than MANUAL_EXPANSION_ORD, the formula derived in Chapter 2 is applied, calculating the necessary coefficients and storing them in the array ord_coef.
- **Conclusion:** After computing the Taylor series coefficients, the function evaluates the series using a truncated power series approach, applying the function fun_taylor. The final result is stored in $c$, the output.

## B.4 Acoth

In this part of the appendix, we present the code used in C to implement the formulas derived for the inverse hyperbolic cotangent (`acoth`) function. The code is briefly commented to focus on the key features and improvements made.

```
1  void
2  FUN(acoth) (const T *a, T *c)                    // checked for real and complex
3  {
4    assert(a && c); DBGFUN(->);
5    ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6    NUM a0 = a->coef[0];
7    ensure(fabs(a0) SELECT(> 1, != 1 && a0 != 0), "invalid domain acoth("FMT")", VAL(a0));
8
9    NUM f0 = atanh(NUMF(inv)(a0));
10   ord_t to = c->mo;
11   if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
12
13 #if OLD_SERIES
14   if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
15     // acoth(x) = 1/2 ln((x+1) / (x-1))
16     T *tn = GET_TMPX(c), *td = GET_TMPX(c);
17     FUN(copy)(a, tn);
18     FUN(seti)(tn, 0, 1, 1);
19     FUN(copy)(a, td);
20     FUN(seti)(td, 0, 1, -1);
21     FUN(logxdy)(tn, td, c);
22     FUN(scl)(c, 0.5, c);
23     REL_TMPX(td), REL_TMPX(tn); DBGFUN(<-); return;
24   }
25 #endif // OLD_SERIES
26
27   NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/(1-a2), f2 = f1*f1, f4 = f2*f2;
28   // fill orders <= MANUAL_EXPANSION_ORD
29   switch(MIN(to, MANUAL_EXPANSION_ORD)) {
30   case 6: ord_coef[6] = a0*(1 + a2*(10./3 + a2)) *f4*f2; /* FALLTHRU */
```

```
31      case 5: ord_coef[5] = (1./5 + a2*(2 + a2)) *f4*f1;      /* FALLTHRU */
32      case 4: ord_coef[4] = a0*(1 + a2) *f4;                  /* FALLTHRU */
33      case 3: ord_coef[3] = (1./3 + a2) *f2*f1;               /* FALLTHRU */
34      case 2: ord_coef[2] = a0 *f2;                           /* FALLTHRU */
35      case 1: ord_coef[1] = f1;                               /* FALLTHRU */
36      case 0: ord_coef[0] = f0;                               break;
37      assert(!"unexpected missing coefficients");
38      }
39      // fill orders > MANUAL_EXPANSION_ORD
40      for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
41        ord_t n = (o-3)/2+1;
42        int s = 2*(o & 1)-1; // o: even = -1, odd = 1
43        NUM v = 0;
44        for (ord_t i = 0; i <= n; ++i, s = -s) {
45          num_t c = s * mad_num_powi(2,o-2*i-1) * mad_num_binom(o-i-1,i);
46          v += c * NUMF(div)(NUMF(powi)(a0,o-2*i-1), NUMF(powi)(a2-1,o-i));
47        }
48        ord_coef[o] = -v/o;
49      }
50
51      fun_taylor(a,c,to,ord_coef);
52      DBGFUN(<-);
53    }
```

- **Initialization and Basic Setup:** The function first computes $f_0 = \text{atanh}(\frac{1}{a_0})$, where $a_0$ is the real part of the dual number given in input. This value is the initial estimate for the inverse hyperbolic cotangent function.

- **Manual Expansion of the Taylor Series:** If the Taylor series order *to* is less than or equal to a predefined limit MANUAL_EXPANSION_ORD, set to 6, the function computes the Taylor coefficients manually. These coefficients are stored in the array ord_coef.

- **Old Approach (OLD_SERIES):** If the series order exceeds MANUAL_EXPANSION_ORD, the computation follows the identity:

$$\text{acoth}(x) = \frac{1}{2}\ln\left(\frac{x+1}{x-1}\right)$$

  This allows for a more direct approach using logarithms for the inverse hyperbolic cotangent.

- **New Approach:** For Taylor series orders greater than MANUAL_EXPANSION_ORD, the formula derived in Chapter 2 is applied, calculating the necessary coefficients and storing them in the array ord_coef.

- **Conclusion:** After computing the Taylor series coefficients, the function evaluates the series using a truncated power series approach, applying the function fun_taylor. The final result is stored in $c$, the output.

## B.5 Acosh

```
void
FUN(acosh) (const T *a, T *c)                    // checked for real and complex
{
  assert(a && c); DBGFUN(->);
  ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
  NUM a0 = a->coef[0];
  ensure(SELECT(a0 > 1, 1), "invalid domain acosh("FMT")", VAL(a0));
  NUM f0 = acosh(a0);

  ord_t to = c->mo;
  if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }

#if OLD_SERIES
  if (to > MANUAL_EXPANSION_ORD) { // use simpler and faster approach?
    // acosh(x) = ln(x + sqrt(x^2-1))
    FUN(logaxpsqrtbpcx2)(a, 1, -1, 1, c);
    DBGFUN(<-); return;
  }
#endif // OLD_SERIES

  NUM ord_coef[to+1], a2 = a0*a0, f1 = 1/sqrt(a2-1), f2 = f1*f1, f4 = f2*f2;
  // fill orders <= MANUAL_EXPANSION_ORD
  switch(MIN(to, MANUAL_EXPANSION_ORD)) {
  case 6: ord_coef[6] = -a0*(5./16 + a2*(5./6 + 1./6*a2)) *f4*f4*f2*f1; /* FALLTHRU */
  case 5: ord_coef[5] = (3./40 + a2*(3./5 + 1./5*a2)) *f4*f4*f1;        /* FALLTHRU */
  case 4: ord_coef[4] = -a0*(3./8 + 1./4*a2) *f4*f2*f1;                 /* FALLTHRU */
  case 3: ord_coef[3] = (1./6 + 1./3*a2) *f4*f1;                        /* FALLTHRU */
  case 2: ord_coef[2] = -a0*(1./2) *f2*f1;                              /* FALLTHRU */
  case 1: ord_coef[1] = f1;                                             /* FALLTHRU */
  case 0: ord_coef[0] = f0;                                             break;
  assert(!"unexpected missing coefficients");
  }
  // fill orders > MANUAL_EXPANSION_ORD
  if (to > MANUAL_EXPANSION_ORD) {
    NUM _a0p1 = 1/(a0+1);
    NUM _a0m1 = 1/(a0-1);
    NUM   den = NUMF(powi)(-2,MANUAL_EXPANSION_ORD-1) * sqrt((a0+1)*(a0-1));

    for (ord_t o = 1+MANUAL_EXPANSION_ORD; o <= to; ++o) {
      int n = (o-1)/2+1;
      NUM num = 0; den *= -2;
      FOR(i,n) {
        int d = o-2*i-1 ? 1 : 2;
        num_t c = (i ? mad_num_fact2(2*i-1) : 1.) / d;
        num += c * mad_num_fact2(2*o-2*i-3) * mad_num_binom(o-1,i) *
              (NUMF(powi)(_a0p1,o-i-1) * NUMF(powi)(_a0m1,i) +
               NUMF(powi)(_a0m1,o-i-1) * NUMF(powi)(_a0p1,i) );
      }
      ord_coef[o] = num/den/mad_num_fact(o);
    }
  }

  fun_taylor(a,c,to,ord_coef);
  DBGFUN(<-);
}
```

- **Initialization:**
  - Extract $a_0$ (the scalar part of $a$) and compute $f_0 = \operatorname{acosh}(a_0)$.
  - Ensure the domain condition $a_0 > 1$. If to, the truncation order, $= 0$ or $a$ is a scalar, set $c$ to

$f_0$ and exit.

- **Old Approach (`OLD_SERIES`):** For to > MANUAL_EXPANSION_ORD, where to is the truncation order, the function uses the simplified formula:

$$\operatorname{acosh}(x) = \ln(x + \sqrt{x^2 - 1}),$$

- **New Approach:**

  - **Taylor Expansion (Orders $\leq$ MANUAL_EXPANSION_ORD):** Precompute:

$$f_1 = \frac{1}{\sqrt{a_0^2 - 1}}, \quad f_2 = f_1^2, \quad f_4 = f_2^2,$$

    and manually calculate coefficients for orders $k = 0, \ldots,$ MANUAL_EXPANSION_ORD using closed-form expressions.

  - **High-Order Terms (Orders > MANUAL_EXPANSION_ORD):** Use a recurrence relation to calculate:

$$\text{ord\_coef}[o] = \frac{\text{num}}{\text{den} \cdot o!},$$

    where num and den correspond to the numerator and denominator in the formula for $\operatorname{acosh}(x)$ in Equation eq. 2.12, computed iteratively.

- **Final Evaluation:** After computing all Taylor series coefficients, evaluate the series using `fun_taylor` and store the result in $c$.

## B.6 Faddeeva function

```
1   void
2   FUN(wf) (const T *a, T *c)
3   {
4     assert(a && c); DBGFUN(->);
5     ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6     NUM a0 = a->coef[0];
7     NUM f0 = NUMF(wf)(a0);
8
9     ord_t to = c->mo;
10    if (!to || FUN(isval)(a)) { FUN(setval)(c,f0); DBGFUN(<-); return; }
11
12    NUM ord_coef[to+1];
13    ord_coef[0] = f0;
14    ord_coef[1] = -2*a0*f0 + I*M_2_SQRTPI;
15
16    if (fabs(a0) <= 7 || (cimag(a0) < creal(a0)+0.5 && cimag(a0) < -creal(a0)+0.5)) {
17      NUM p[to+1], q[to+1];
18      p[0] = 1;
```

```
19        p[1] = -2*a0;
20        q[0] = 0;
21        q[1] = 1;
22
23        for (ord_t o = 2; o <= to; ++o) {
24          q[o] = 0;
25    #ifdef MAD_CTPSA_IMPL
26          q[o] = -2*(o-1)*q[o-2] - 2*a0*q[o-1];
27    #endif
28          p[o] = -2*(o-1)*p[o-2] - 2*a0*p[o-1];
29          ord_coef[o] = (p[o]*f0 + q[o]*I*M_2_SQRTPI)/mad_num_fact(o);
30        }
31
32      } else if (fabs(a0) > 7 && fabs(a0) < 100) { // adjusted for double precision
33    #ifdef MAD_CTPSA_IMPL
34        NUM a02 = a0*a0, a04=a02*a02, a06=a04*a02, a08=a04*a04;
35        NUM coef[7] = {
36          -I*M_SQRTPI/(a04),
37          -I*M_SQRTPI/(a06)       *5,
38          -I*M_SQRTPI/(a06*a02)/4 *105,
39          -I*M_SQRTPI/(a06*a04)/4 *630,
40          -I*M_SQRTPI/(a08*a04)/16*17325,
41          -I*M_SQRTPI/(a08*a06)/16*135135,
42          -I*M_SQRTPI/(a08*a08)/64*4729725 };
43    #endif
44
45
46        ord_coef[2] = -a0*(-2*a0*f0 + I*M_2_SQRTPI) - f0;
47
48        for (ord_t o = 3; o <= to; ++o) {
49          FOR(i,7) ord_coef[o] += coef[i];
50          FOR(i,7) coef[i] *= -1/a0*(2.*i+o+1)/(o+1);
51        }
52      } else {
53        for (ord_t o = 1; o <= to; ++o)
54          ord_coef[o] = -ord_coef[o-1]/a0;
55      }
56
57      fun_taylor(a,c,to,ord_coef);
58      DBGFUN(<-);
59    }
```

- **Initialization and Basic Setup:** The function begins by extracting the zeroth-order coefficient $a_0 = a \to \text{coef}[0]$ and computing $f_0 = \text{wf}(a_0)$ (wf here is the name of the Faddeeva function). If the truncation order **to** $= 0$ or the input $a$ is a scalar, the output series $c$ is set to $f_0$, and the function exits.

- **First Branch (Exact formula eq. 2.15)** When $|a_0| \leq 7$ or $\text{Im}(a_0) < -|\text{Re}(a_0)| + 0.3$, a direct Taylor expansion is computed using eq. 2.15:

$$p_0 = 1, \quad p_1 = -2a_0, \quad p_n = -2(n-1)p_{n-2} - 2a_0 p_{n-1},$$

$$q_0 = 0, \quad q_1 = 1, \quad q_n = -2(n-1)q_{n-2} - 2a_0 q_{n-1}.$$

The coefficients are derived as:

$$\text{ord\_coef}[n] = \frac{p_n f_0 + q_n \frac{2i}{\sqrt{\pi}}}{n!}.$$

- **Second Branch (Perturbative Approach):** When $7 < |a_0| < 100$, we use the perturbative approach (plus an exponential decaying correction; we did not present it before because it is not significant and we reach soon the underflow regime). The coefficients are computed using a combination of precomputed terms and numerical adjustments. Specifically:

  the Taylor coefficients are evaluated as:

  $$\text{ord\_coef}[n] = \sum_k \text{coef}[k],$$

  where the precomputed terms $\text{coef}[k]$ are iteratively updated.

- **Third Branch (Simplified Perturbative Approach):** For $|a_0| \geq 100$, the computation is simplified. Coefficients are determined via:

  $$\text{ord\_coef}[n] = -\frac{\text{ord\_coef}[n-1]}{a_0}.$$

  This reduces complexity while maintaining stability for large $|a_0|$. In this scenario we keep only the first order term of the perturbative approach; this is possible thank to the fast decaying of the higher order terms.

- **Finalization:** The computed coefficients ord_coef are used to evaluate the Taylor series via fun_taylor, and the result is stored in $c$, completing the computation.

## B.7 Sinc

```
1  void
2  FUN(sinc) (const T *a, T *c)
3  {
4    assert(a && c); DBGFUN(->);
5    ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6
7    NUM a0 = a->coef[0];
8    NUM f0 = NUMF(sinc)(a0);
9    ord_t to = c->mo;
10
11   if (!to || FUN(isval)(a)) {
12     FUN(setval)(c,f0); DBGFUN(<-); return;
13   }
14
15   if (fabs(a0) > 0.75) { // sinc(x), |x| > 0.75
16     T *t = GET_TMPX(c);
17     FUN(sin)(a,t);
18     FUN(div)(t,a,c);
19     FUN(seti)(c,0,0,f0); // scalar part not very accurate
20     REL_TMPX(t); DBGFUN(<-); return;
21   }
22
23   NUM ord_coef[to+1];
```

```
24
25    if (fabs(a0) > 1e-12) { // sinc(x), 1e-12 < |x| <= 0.75
26      num_t odd_coef[7] = {
27        -3, 30, -840, 45360, -3991680, 518918400, -93405312000
28      };
29      int s = 1; // o/2: even = 1, odd = -1
30      num_t fact = 1;
31      for (ord_t o = 1; o <= to; o += 2, s = -s) {
32        NUM v1 = 0, v2 = 0;
33        FOR(i,7) {
34          v1 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i+1);
35          v2 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i)*(2*i+1);
36        }
37        fact *= o*(o+1.);
38        ord_coef[ o          ] = s * v1 / fact*(o+1);
39        ord_coef[(o+1)%(to+1)] = s * v2 / fact;
40        if (o+2 <= to) {
41          static const num_t stp_coef[7] = {
42            -2, 12, -240, 10080, -725760, 79833600, -12454041600
43          };
44          FOR(i,7) odd_coef[i] += stp_coef[i];
45        }
46      }
47      ord_coef[0] = f0;
48    } else { // sinc(x), |x| <= 1e-12
49      ord_coef[0] = 1;
50      ord_coef[1] = 0;
51      for (ord_t o = 2; o <= to; ++o)
52        ord_coef[o] = -ord_coef[o-2] / (o*(o+1.));
53    }
54
55    fun_taylor(a,c,to,ord_coef);
56    DBGFUN(<-);
57  }
```

## Key Variables and Arrays

- $a_0$**:** Zeroth coefficient of $a$, representing the point at which the Taylor series is expanded.
- **ord_coef**$[i]$**:** Array storing the Taylor series coefficients for each order $i \in \{0, \ldots, \text{to}\}$.
- **odd_coef**$_0$**:** Auxiliary array of coefficients.

$$\text{odd\_coef}_0 = [-3, 30, -840, 45360, -3991680, 518918400, -93405312000].$$

- **add_odd:** Incremental coefficients for updating odd_coef:

$$\text{add\_odd} = [2, 12, -240, -10080, -725760, -79833600, -12454041600].$$

### Case Analysis

**1. Large $|a_0|$ ($|a_0| > 0.75$)**

In this regime, the function directly computes $\text{sinc}(x)$ using its definition:

$$\text{sinc}(x) = \frac{\sin(x)}{x}.$$

- Compute $\sin(a)$ and store the result in a temporary variable $t$.
- Divide $t$ by $a$ to obtain $\text{sinc}(a)$, and store this in $c$.
- Adjust the scalar part ($c[0]$) to match $f_0 = \text{sinc}(a_0)$.

**2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.75$)**

This is the most complex case, where the Taylor series expansion is computed for moderate values of $|a_0|$:

1. Initialize $\text{odd\_coef}_0$ for odd derivatives of $\text{sinc}(x)$.
2. Loop through each order $o$ up to to, with step size 2.
    - Compute $v_1$ (coefficient contribution for odd derivatives) and $v_2$ (even derivatives).

$$v_1 = \sum_{i=0}^{6} \frac{1}{\text{odd\_coef}[i]} \cdot a_0^{2i+1}, \quad v_2 = \sum_{i=0}^{6} \frac{1}{\text{odd\_coef}[i]} \cdot a_0^{2i} \cdot (2i+1).$$

    - Update $\text{ord\_coef}[o]$ and $\text{ord\_coef}[o+1]$:

$$\text{ord\_coef}[o] = \frac{(-1)^{o/2} \cdot v_1}{o!} * (o+1), \quad \text{ord\_coef}[o+1] = \frac{(-1)^{o/2} \cdot v_2}{o!}.$$

    - Increment odd_coef using add_odd.
3. Set the zeroth coefficient to $f_0 = \text{sinc}(a_0)$.

**3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)**

For very small values of $|a_0|$, $\mathrm{sinc}(x)$ is approximated using the series expansion:

$$\mathrm{sinc}(x) \approx 1 - \frac{x^2}{6} + \frac{x^4}{120} - \cdots$$

1. Set $\mathrm{ord\_coef}[0] = 1$ and $\mathrm{ord\_coef}[1] = 0$.
2. Compute higher-order coefficients recursively:

$$\mathrm{ord\_coef}[o] = -\frac{\mathrm{ord\_coef}[o-2]}{o \cdot (o+1)}.$$

### Final Evaluation

Finally, the Taylor series is evaluated using the function `fun_taylor`, which combines the computed coefficients ord_coef to generate the output $c$.

## B.8 Sinhc

```
1  void
2  FUN(sinhc) (const T *a, T *c)
3  {
4    assert(a && c); DBGFUN(->);
5    ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");
6
7    NUM a0 = a->coef[0];
8    NUM f0 = NUMF(sinhc)(a0);
9    ord_t to = c->mo;
10
11   if (!to || FUN(isval)(a)) {
12     FUN(setval)(c,f0); DBGFUN(<-); return;
13   }
14
15   if (fabs(a0) > 0.75) { // sinhc(x), |x| > 0.75
16     T *t = GET_TMPX(c);
17     FUN(sinh)(a,t);
18     FUN(div)(t,a,c);
19     FUN(seti)(c,0,0,f0); // scalar part not very accurate
20     REL_TMPX(t); DBGFUN(<-); return;
21   }
22
23   NUM ord_coef[to+1];
24   if (fabs(a0) > 1e-12) { // sinhc(x), 1e-12 < |x| <= 0.75
25     num_t odd_coef[7] = {
26       3, 30, 840, 45360, 3991680, 518918400, 93405312000
27     };
28     num_t fact = 1;
29     for (ord_t o = 1; o <= to; o += 2) {
30       NUM v1 = 0, v2 = 0;
31       FOR(i,7) {
```

```
32          v1 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i+1);
33          v2 += 1 / odd_coef[i] * NUMF(powi)(a0,2*i)*(2*i+1);
34        }
35        fact *= o*(o+1.);
36        ord_coef[ o          ] = v1 / fact*(o+1);
37        ord_coef[(o+1)%(to+1)] = v2 / fact;
38        if (o+2 <= to) {
39          static const num_t stp_coef[7] = {
40            2, 12, 240, 10080, 725760, 79833600, 12454041600
41          };
42          FOR(i,7) odd_coef[i] += stp_coef[i];
43        }
44      }
45      ord_coef[0] = f0;
46    } else { // sinhc(x), |x| <= 1e-12
47      ord_coef[0] = 1;
48      ord_coef[1] = 0;
49      for (ord_t o = 2; o <= to; ++o)
50        ord_coef[o] = ord_coef[o-2] / (o*(o+1.));
51    }
52
53    fun_taylor(a,c,to,ord_coef);
54    DBGFUN(<-);
55  }
```

## Key Variables and Arrays

- $a_0$**:** Zeroth coefficient of $a$, representing the point at which the Taylor series is expanded.
- **ord_coef**$[i]$**:** Array storing the Taylor series coefficients for each order $i \in \{0, \ldots, \text{to}\}$.
- **odd_coef**$_0$**:** Auxiliary array of coefficients.

$$\text{odd\_coef}_0 = [3, 30, 840, 45360, 3991680, 518918400, 93405312000].$$

- **add_odd:** Incremental coefficients for updating odd_coef:

$$\text{add\_odd} = [2, 12, 240, 10080, 725760, 79833600, 12454041600].$$

## Case Analysis

### 1. Large $|a_0|$ ($|a_0| > 0.75$)

In this regime, the function directly computes $\text{sinhc}(x)$ using its definition:

$$\text{sinhc}(x) = \frac{\sinh(x)}{x}.$$

- Compute $\sinh(a)$ and store the result in a temporary variable $t$.

- Divide $t$ by $a$ to obtain $\text{sinhc}(a)$, and store this in $c$.
- Adjust the scalar part ($c[0]$) to match $f_0 = \text{sinhc}(a_0)$.

**2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.75$)**

This is the most complex case, where the Taylor series expansion is computed for moderate values of $|a_0|$:

1. Initialize $\text{odd\_coef}_0$ for odd derivatives of $\text{sinhc}(x)$.
2. Loop through each order $o$ up to to, with step size 2.
   - Compute $v_1$ (coefficient contribution for odd derivatives) and $v_2$ (even derivatives).

$$v_1 = \sum_{i=0}^{6} \frac{1}{\text{odd\_coef}[i]} \cdot a_0^{2i+1}, \quad v_2 = \sum_{i=0}^{6} \frac{1}{\text{odd\_coef}[i]} \cdot a_0^{2i} \cdot (2i+1).$$

   - Update $\text{ord\_coef}[o]$ and $\text{ord\_coef}[o+1]$:

$$\text{ord\_coef}[o] = \frac{v_1}{o!} * (o+1), \quad \text{ord\_coef}[o+1] = \frac{v_2}{o!}.$$

   - Increment odd_coef using add_odd.
3. Set the zeroth coefficient to $f_0 = \text{sinhc}(a_0)$.

**3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)**

For very small values of $|a_0|$, $\text{sinhc}(x)$ is approximated using the series expansion:

$$\text{sinhc}(x) \approx 1 + \frac{x^2}{6} + \frac{x^4}{120} + \cdots$$

1. Set $\text{ord\_coef}[0] = 1$ and $\text{ord\_coef}[1] = 0$.
2. Compute higher-order coefficients recursively:

$$\text{ord\_coef}[o] = \frac{\text{ord\_coef}[o-2]}{o \cdot (o+1)}.$$

### Final Evaluation

Finally, the Taylor series is evaluated using the function `fun_taylor`, which combines the computed coefficients ord_coef to generate the output $c$.

## B.9 Asinc

```
void
FUN(asinc) (const T *a, T *c)
{
  assert(a && c); DBGFUN(->);
  ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");

  NUM a0 = a->coef[0];
  NUM f0 = NUMF(asinc)(a0);
  ord_t to = c->mo;

  if (!to || FUN(isval)(a)) {
    FUN(setval)(c,f0); DBGFUN(<-); return;
  }

  if (fabs(a0) > 0.58) { // asinc(x), |x| > 0.58
    T *t = GET_TMPX(c);
    FUN(asin)(a,t);
    FUN(div)(t,a,c);
    FUN(seti)(c,0,0,f0); // scalar part not very accurate
    REL_TMPX(t); DBGFUN(<-); return;
  }

  NUM ord_coef[to+1];

  if (fabs(a0) > 1e-12) { // asinc(x), 1e-12 < |x| <= 0.58
    FOR(i,to+1) ord_coef[i] = 0;

    enum { ord = DESC_MAX_ORD+1 };
    static num_t scl_coef[ord+1] = {0};
    if (!scl_coef[0]) {
      scl_coef[0] = 1./3;
      FOR(i,1,ord+1) scl_coef[i] = scl_coef[i-1] * SQR(2*i+1.)/(i*(4*i+6.));
    }

    num_t fact = 1;
    num_t tmp_coef[ord+1]; FOR(i,ord+1) tmp_coef[i] = scl_coef[i];
    for (ord_t o = 1; o <= to; o += 2) {
      NUM a0pi = 1, dc1, dc2;
      fact *= o*(o+1.);
      FOR(i,ord+1) {
        tmp_coef[i] *= o > 1 ? CUB(2.*i+o) / (2*i+o+2) : 1;
        ord_coef [ o          ] += (dc1=a0*SQR(a0pi)*tmp_coef[i]*(  o+1) / fact);
        ord_coef [(o+1)%(to+1)] += (dc2=   SQR(a0pi)*tmp_coef[i]*(2*i+1) / fact);
        if (fabs(dc1)/fabs(ord_coef[o])            < SQR(DBL_EPSILON) &&
            fabs(dc2)/fabs(ord_coef[(o+1)%(to+1)]) < SQR(DBL_EPSILON)) {
          if (DBG_SERIES)
            printf("asinc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
          break;
        } else if (i == ord) {
          if (DBG_SERIES)
            printf("asinc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
          trace(1, "asinc did not converge after %d iterations", i);
        }
        a0pi *= a0;
      }
```

```
    }
    ord_coef[0] = f0;
  } else { // asinc(x), |x| <= 1e-12
    ord_coef[0] = 1;
    ord_coef[1] = 0;
    for (ord_t o = 2; o <= to; ++o)
      ord_coef[o] = (ord_coef[o-2] * SQR(o-1.)) / (o*(o+1.));
  }

  fun_taylor(a,c,to,ord_coef);
  DBGFUN(<-);
}
```

## Key Variables and Arrays

- $a_0$: The point where the function is evaluated.
- `to`: The maximum order of the Taylor series expansion.
- `ord_coef`: Array storing the coefficients of the Taylor series.
- `tmp_coef`: Temporary coefficients used in the series computation.
- mult: A multiplicative factor dependent on $i$ and $o$:

$$\text{mult} = \begin{cases} 1, & \text{if } o = 1 \\ \frac{(2i+o)^3}{(2i+o+2)}, & \text{otherwise} \end{cases}$$

## Case Analysis

### 1. Large $|a_0|$ ($|a_0| > 0.58$)

For large absolute values of $a_0$:

- Compute $\text{asin}(a)$ directly in the dual space.
- Calculate $f(a) = \frac{\text{asin}(a)}{a}$.
- Set the scalar part of the result (need to reset because of poor accuracy).

### 2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.58$)

For moderate values of $|a_0|$, the computation involves:

- Initializing temporary coefficients `tmp_coef`.
- Computing coefficients for odd and even derivatives:

$$\frac{Asinc^{2o+1}(a_0)}{(2o+1)!} = \text{ord\_coef}[2o+1] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i+1} \cdot \text{tmp\_coef}[i] \cdot \text{mult}}{(2o+1)!}$$

$$\frac{Asinc^{2o+2}(a_0)}{(2o+1)!} = \text{ord\_coef}[2o] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i} \cdot \text{tmp\_coef}[i] \cdot (2i+1) \cdot \text{mult}}{(2o)!}$$

- Iterative update of tmp_coef using the recurrence relation:

$$\text{tmp\_coef}[i] = \text{tmp\_coef}[i-1] \cdot \frac{(2i+1)^2}{i(4i+6)} \quad \text{for } i \geq 1$$

- Stopping computation when contributions become negligibly small.

**3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)**

For very small values of $|a_0|$ we just use the McLaurin expansion of asinc:

- Initialize $\text{ord\_coef}[0] = 1$ and $\text{ord\_coef}[1] = 0$.
- Compute higher-order coefficients using the recurrence relation:

$$\text{ord\_coef}[o] = -\frac{\text{ord\_coef}[o-2]}{o \cdot (o+1)}$$

### Final Evaluation

The Taylor series is evaluated and given as output.

In the old implementation everything was computed using directly $\text{asinc}(x) = \frac{\text{asin(x)}}{x}$

## B.10 Asinhc

```
1  void
2  FUN(asinhc) (const T *a, T *c)
3  {
4    assert(a && c); DBGFUN(->);
```

```
  ensure(IS_COMPAT(a,c), "incompatibles GTPSA (descriptors differ)");

  NUM a0 = a->coef[0];
  NUM f0 = NUMF(asinhc)(a0);
  ord_t to = c->mo;

  if (!to || FUN(isval)(a)) {
    FUN(setval)(c,f0); DBGFUN(<-); return;
  }

  if (fabs(a0) > 0.62) { // asinhc(x), |x| > 0.62
    T *t = GET_TMPX(c);
    FUN(asinh)(a,t);
    FUN(div)(t,a,c);
    FUN(seti)(c,0,0,f0); // scalar part not very accurate
    REL_TMPX(t); DBGFUN(<-); return;
  }

  NUM ord_coef[to+1];

  if (fabs(a0) > 1e-12) { // asinhc(x), 1e-12 < |x| <= 0.62
    FOR(i,to+1) ord_coef[i] = 0;

    enum { ord = DESC_MAX_ORD+1 };
    static num_t scl_coef[ord+1] = {0};
    if (!scl_coef[0]) {
      scl_coef[0] = -1./3;
      FOR(i,1,ord+1) scl_coef[i] = -scl_coef[i-1] * SQR(2*i+1.)/(i*(4*i+6.));
    }

    num_t fact=1;
    num_t tmp_coef[ord+1]; FOR(i,ord+1) tmp_coef[i] = scl_coef[i];
    for (ord_t o = 1; o <= to; o += 2) {
      NUM a0pi = 1, dc1, dc2;
      fact *= o*(o+1.);
      FOR(i,ord+1) {
        tmp_coef[i] *= o > 1 ? -CUB(2.*i+o) / (2*i+o+2) : 1;
        ord_coef [ o          ] += (dc1=a0*SQR(a0pi)*tmp_coef[i]*(  o+1) / fact);
        ord_coef [(o+1)%(to+1)] += (dc2=   SQR(a0pi)*tmp_coef[i]*(2*i+1) / fact);
        if (fabs(dc1)/fabs(ord_coef[o])            < SQR(DBL_EPSILON) &&
            fabs(dc2)/fabs(ord_coef[(o+1)%(to+1)]) < SQR(DBL_EPSILON)) {
          if (DBG_SERIES)
            printf("asinhc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
          break;
        } else if (i == ord) {
          if (DBG_SERIES)
            printf("asinhc: i=%d, o=%d, to=%d, |a0|=%.16e, |dc|=%.16e\n", i, o, to, fabs(a0), MAX(fabs(dc1),fabs(dc2)));
          trace(1, "asinhc did not converged after %d iterations", i);
        }
        a0pi *= a0;
      }
    }
    ord_coef[0] = f0;
  } else { // asinhc(x), |x| <= 1e-12
    ord_coef[0] = 1;
    ord_coef[1] = 0;
    for (ord_t o = 2; o <= to; ++o)
      ord_coef[o] = -(ord_coef[o-2] * SQR(o-1.)) / (o*(o+1.));
  }

  fun_taylor(a,c,to,ord_coef);
  DBGFUN(<-);
}
```

## Key Variables and Arrays

- $a_0$: The point where the function is evaluated.

- `to`: The maximum order of the Taylor series expansion.
- `ord_coef`: Array storing the coefficients of the Taylor series.
- `tmp_coef`: Temporary coefficients used in the series computation.
- mult: A multiplicative factor dependent on $i$ and $o$:

$$\text{mult} = \begin{cases} 1, & \text{if } o = 1 \\ \frac{(2i+o)^3}{(2i+o+2)}, & \text{otherwise} \end{cases}$$

## Case Analysis

**1. Large $|a_0|$ ($|a_0| > 0.62$)**

For large absolute values of $a_0$:

- Compute $\text{asinh}(a)$ directly in the dual space.
- Calculate $f(a) = \frac{\text{asinh}(a)}{a}$.
- Set the scalar part of the result (need to reset because of poor accuracy).

**2. Moderate $|a_0|$ ($1 \times 10^{-12} < |a_0| \leq 0.62$)**

For moderate values of $|a_0|$, the computation involves:

- Initializing temporary coefficients `tmp_coef`.
- Computing coefficients for odd and even derivatives:

$$\frac{Asinhc^{2o+1}(a_0)}{(2o+1)!} = \text{ord\_coef}[2o+1] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i+1} \cdot \text{tmp\_coef}[i] \cdot \text{mult}}{(2o+1)!}$$

$$\frac{Asinhc^{2o+2}(a_0)}{(2o+1)!} = \text{ord\_coef}[2o] = \sum_{i=0}^{\text{ord}} \frac{a_0^{2i} \cdot \text{tmp\_coef}[i] \cdot (2i+1) \cdot \text{mult}}{(2o)!}$$

- Iterative update of tmp_coef using the recurrence relation:

$$\text{tmp\_coef}[i] = \text{tmp\_coef}[i-1] \cdot \frac{(2i+1)^2}{i(4i+6)} \quad \text{for } i \geq 1$$

- Stopping computation when contributions become negligibly small.

**3. Small $|a_0|$ ($|a_0| \leq 1 \times 10^{-12}$)**

For very small values of $|a_0|$ we just use the McLaurin expansion of asinhc:

- Initialize ord_coef$[0] = 1$ and ord_coef$[1] = 0$.
- Compute higher-order coefficients using the recurrence relation:

$$\text{ord\_coef}[o] = -\frac{\text{ord\_coef}[o-2]}{o \cdot (o+1)}$$

## Final Evaluation

The Taylor series is evaluated and given as output.

In the old implementation everything was computed using directly $\text{asinhc}(x) = \frac{\text{asinh(x)}}{x}$